Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации
федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

09.03.01 Информатика и вычислительная техника
(направление подготовки/специальность)
Программное обеспечение мобильных систем
(профиль/специализация)
Очная
(форма обучения)

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

(вид практики)

Тип практики Технологическая (проектно-технологическая) практика
на предприятии ООО «Бюро 1440»
(наименование профильной организации/структурного подразделения СибГУТИ)

ТЕМА ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

Разработка системы модуляции и демодуляции QPSK и QAM сигналов на платформе SDR

Выполнил:
студент института информатики и вычислительной техники Любимов Кирилл Алексеевич
группа ИА-331

_____ / Любимов К. А. /
(подпись) *(ФИО)*

«20» июля 2026 г.

Проверил:
Руководитель практики от СибГУТИ _____ / Брагин К. И. /
(подпись) *(ФИО)*

«20» июля 2026 г.

отметка [1] отлично _____ «20» июля 2026 г.

Новосибирск 2026

[1] Заполняется во время промежуточной аттестации

НАПРАВЛЕНИЕ НА ПРАКТИКУ

На основе договора[2] № 2025/97/БР-Д-25-2074 от 26.11.2025
федеральное государственное бюджетное образовательное учреждение высшего образования "Сибирский государственный университет телекоммуникаций и информатики" (СибГУТИ)
направляет студента 3 курса, института информатики и вычислительной техники, гр ИА-331
Любимова Кирилла Алексеевича
для прохождения технологической практики с 03.06.2026 по 20.07.2026 в/на ООО «Бюро 1440»

МП Директор института[3] _____ / _____

ОХРАНА ТРУДА[3]

Вводный инструктаж технический писатель, Андреев А.В.
(должность, Ф.И.О.)

03.06.2026 _____
(подпись)

М.П.

Зачет принял технический писатель, Андреев А.В.
(должность, Ф.И.О.)

03.06.2026 _____
(подпись)

М.П.

[2] При прохождении практики на кафедре оставить пустым.
[3] Вводный инструктаж проводится по месту прохождения практики.

План-график проведения практики

Направление: 09.03.01 Информатика и вычислительная техника
(код, наименование направления (специальности))
Направленность (профиль)/ специализация: Программное обеспечение мобильных систем
Объем практики: 360/10 часов/ЗЕ
Тип практики: Технологическая (проектно-технологическая) практика

Содержание практики[4]:
Тема индивидуального задания практики Разработка системы модуляции и демодуляции QPSK и QAM сигналов на платформе SDR

Наименование видов деятельности в соответствии с рабочей программой практики	Дата (начало – окончание)
Подготовительный: Общее ознакомление со структурным подразделением предприятия, вводный инструктаж по технике безопасности Выдача задания на практику, деление студентов на группы (если необходимо), определение конкретной индивидуальной темы, формирование плана работ	03.06 – 06.06
Основной: Работа с библиотечными фондами структурного подразделения или предприятия, сбор и анализ материалов по теме практики Выполнение работ в соответствии с составленным планом	07.06 – 18.07
Итоговый: Анализ полученных результатов и произведённой работы, составление отчета по практике	19.07, 20.07

Руководитель практики
от профильной организации*
«03» июня 2026 г. _____ / Андреев А. В. /
(подпись) *(ФИО)*

Руководитель практики от СибГУТИ
«03» июня 2026 г. _____ / Брагин К. И. /
(подпись) *(ФИО)*

Студент
« 20 » июня 2026 г. _____ / Любимов К. А./
(подпись) *(ФИО)*

[4] В случае прохождения практики в профильной организации

КРАТКАЯ ХАРАКТЕРИСТИКА РАБОТЫ ПРАКТИКАНТА

Отзыв руководителя от предприятия о работе студента (выполнение программы практики, овладение производственными навыками, отношение к работе, трудовая дисциплина и др.)[6]

Студент Любимов К. А. успешно прошел технологическую практику, проявив хорошие знания в области SDR, навыки программирования и работы с оборудованием Adalm Pluto. Ответственно относился к выполнению заданий, проявил интерес к теме, внимание к деталям и способность к самостоятельной работе.

Руководитель практики от предприятия: _____ / _____ М.П.

Отзыв руководителя от СибГУТИ о работе студента (выполнение программы практики, овладение производственными навыками, отношение к работе, трудовая дисциплина и др.)

Студент Любимов К.А успешно прошёл технологическую практику, проявил хорошие знания в области SDR, ЦОС, навыки программирования и архитектура Adalm Pluto, проявил интерес к теме и ответственно отнесся к поставленной задаче

Компетенции	Уровень сформированности компетенций[7]
ПК-1 Способен разрабатывать требования и проектировать программное обеспечение	высокий
ПК-3 Способен осуществлять эксплуатацию и развитие транспортных сетей и сетей передачи данных, включая спутниковые системы	высокий

отметка о зачете (оценка) отлично

Руководитель практики от СибГУТИ: _____ / Брагин К. И. /

[6] Оставить пустым в случае прохождения практики на кафедре.

[7] Уровень сформированности компетенций: высокий, средний, низкий, не сформирована.

СОДЕРЖАНИЕ

1 ПРАКТИКА: АРХИТЕКТУРА SDR СИСТЕМЫ 3

2 ЗНАКОМСТВО С БИБЛИОТЕКАМИ SOAPY SDR, LIBIIO 11

3 РАБОТА С БИБЛИОТЕКАМИ SOAPY SDR, LIBIIO. 20

4 ИМИТАЦИЯ АНАЛОГОВОЙ ПЕРЕДАЧИ ЗВУКА И ЕГО ПРИЕМ С ИСПОЛЬЗОВАНИЕМ SDR. 24

5 ДИСКРЕТНАЯ СВЕРТКА. РЕАЛИЗАЦИЯ ПРИЕМА И ПЕРЕДАЧИ BPSK СИГНАЛОВ 28

6 ПРИЕМ QPSK BPSK, ПРИЕМ НА СОГЛАСОВАННЫЙ ФИЛЬТР, ГЛАЗКОВАЯ ДИАГРАММА, ПОИСК ОПТИМАЛЬНОГО ОТСЧЕТНОГО ЗНАЧЕНИЯ 35

7 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ДЕТЕКТОРА ВРЕМЕННОЙ ОШИБКИ(СИНХРОНИЗАЦИЯ ПРИЕМНИКА И ПЕРЕДАТЧИКА) НА SDR 42

8 РЕАЛЗИАЦИЯ СИМВОЛЬНОЙ СИНХРОНИЗАЦИИ. ПРИМЕНЕНИЕ К СЭМПЛАМ С ADALMPLUTO. РАЗРАБОТКА ГРАФИЧЕСКОГО ИНТЕРФЕЙСА (GUI) ДЛЯ РАБОТЫ С SDR. ОКНА НАСТРОЕК SDR, ВЫВОД РЕЗУЛЬТАТОВ ОБРАБОТКИ СЭМПЛОВ И Т.Д. ... 50

9 КОРРЕКЦИЯ ЧАСТОТНОГО СМЕЩЕНИЯ В ОТСЧЁТАХ ПРИНЯТОГО СИГНАЛА ПЕРЕХОД ОТ ОДНОЙ НЕСУЩЕЙ К OFDM , НАЧАЛО ФОРМИРОВАНИЯ И ПРИЕМА OFDM 56

10 ФОРМИРОВАНИЯ И ПРИЕМ OFDM. ПОСТРОЕНИЕ СПЕКТРА РАДИОСИГНАЛА (ПРЕОБРАЗОВАНИЕ ФУРЬЕ). ИЗУЧЕНИЕ РАБОТЫ БИБЛИОТЕКИ FFTW3 63

11 РЕАЛИЗАЦИЯ КОРРЕКЦИИ ЧАСТОТНОГО СМЕЩЕНИЯ ПО ПИЛОТАМ..... 67

1 ПРАКТИКА
АРХИТЕКТУРА SDR СИСТЕМЫ
УСТАНОВКА ПО, НАСТРОЙКА УСТРОЙСТВА

Ссылка на GitHub

Цель практики:

Узнать, что такое SDR, изучить принципы его работы и внутреннюю архитектуру на базовом уровне. Познакомиться с инструментом GNU Radio и создать с его помощью программу для SDR, позволяющую принимать радио.

Краткие теоретические сведения

Что такое SDR?

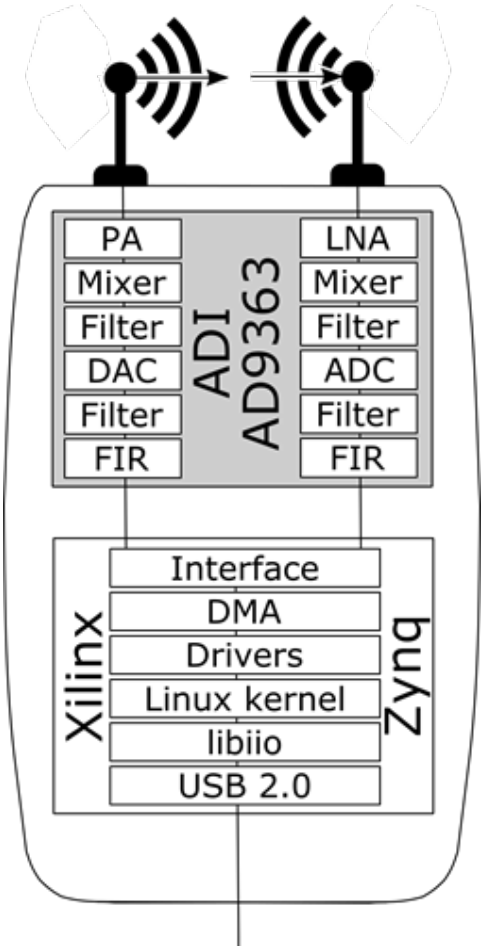


Рис. 1 — ADALM Pluto

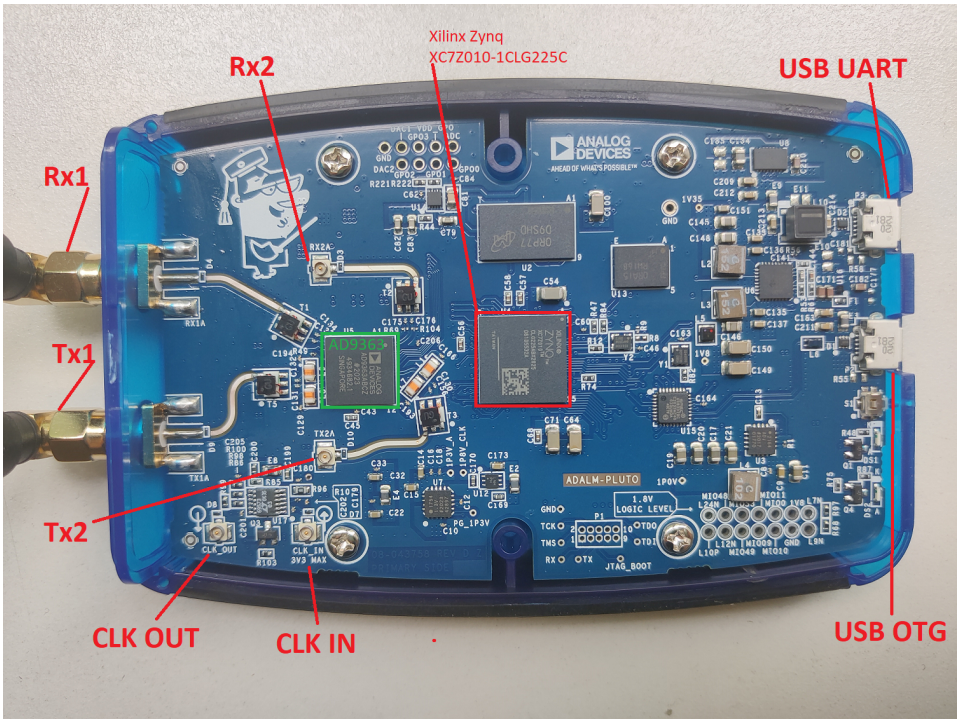


Рис. 2 — ADALM Pluto

Обучающая платформа PlutoSDR может взаимодействовать с:

1. Matlab, Simulink
2. GNU Radio
3. C, C++ при помощи дополнительных библиотек
4. C#
5. Среда языка Python

В начале данного курса мы наладим взаимодействие PlutoSDR с языком Python. Далее напишем программы под другие платформы, сравним разницу по времени обработки сигналов, простоте написания кода и редактирования.

Чип AD9363

Программируемый РЧ приемопередатчик, возможности которого позволяют использовать его для построения микро- (фемто-) сот мобильной связи 3G, 4G и 5G (в некоторых конфигурациях).

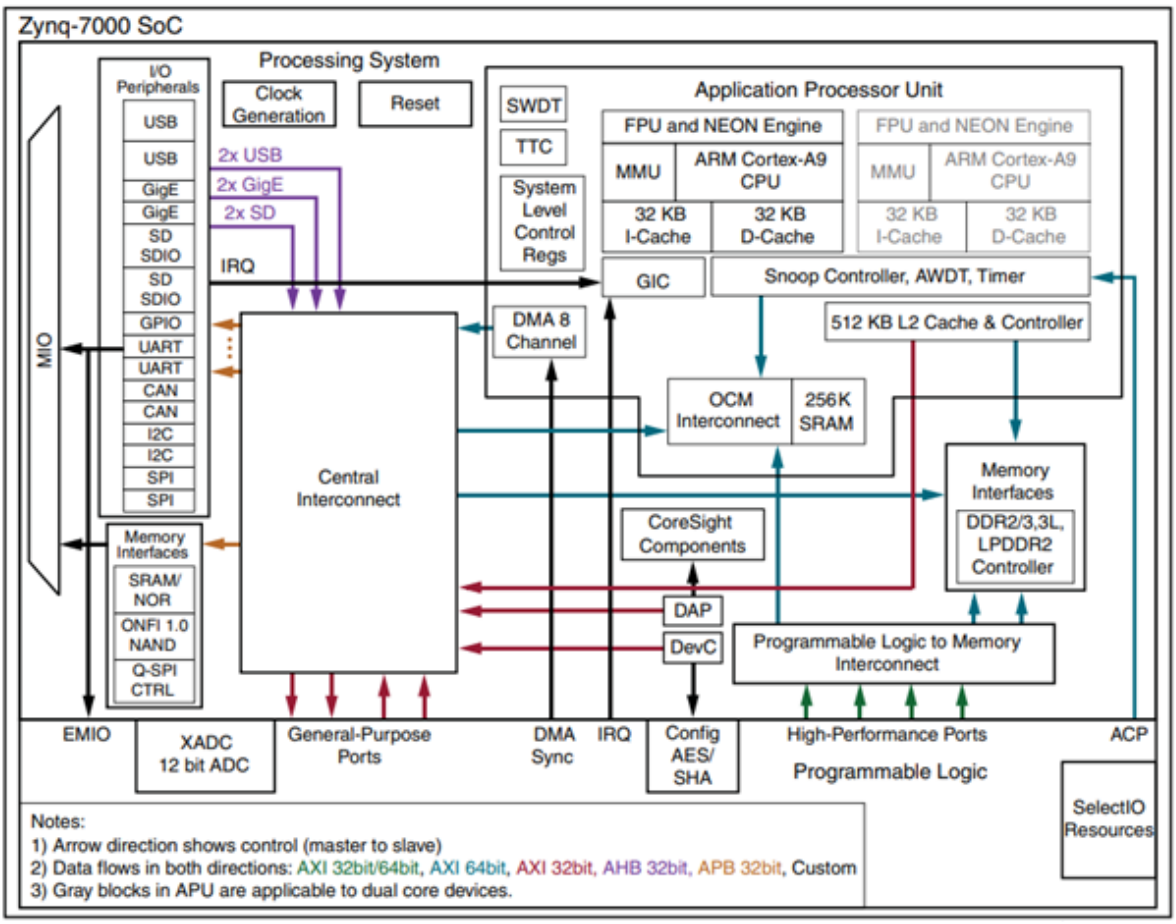


Рис. 3 — ПЛИС Xilinx Zynq

Основные характеристики

- **12-битный** ЦАП/АЦП (Цифро-аналоговый/Аналого-цифровой Преобразователь)
- Поддерживаемые несущие частоты от 90 [МГц] до **3.8** [ГГц]
- Поддерживает временной и частотный дуплексы (**TDD**, **FDD**)
- Ширина полосы частот: **20** [МГц]
- Шумы в приемнике: **3** [dB]
- EVM (Error Vector Magnitude): **-34** [dB]
- Tx noise: **< -157** [dBm/Hz]
- 2 **Rx**, 2 **Tx**

Структурная схема Zynq

Каждый Zynq состоит из одного или двух ядер ARM Cortex-A9 (ARM v7), кэш L1 у каждого ядра свой, кэш L2 общий. Поддерживаемая оперативная память имеет стандарты DDR3, DDR3L, DDR2, LPDDR-2. Максимальный объем оперативной памяти равен 1 Гбайт (2 микросхемы по 4 Гбит). Максимальная тактовая частота оперативной памяти 525 МГц. Операционные системы: Standalone

(bare-metal) и Petalinux. Процессорный модуль общается с внешним миром и программируемой логикой с помощью портов, объединенных в группы:

- MIO (multiplexed I/O)
- EMIO (extended multiplexed I/O)
- GP (General-Purpose Ports)
- HP (High-Performance Ports)
- ACP (Accelerator coherency port)

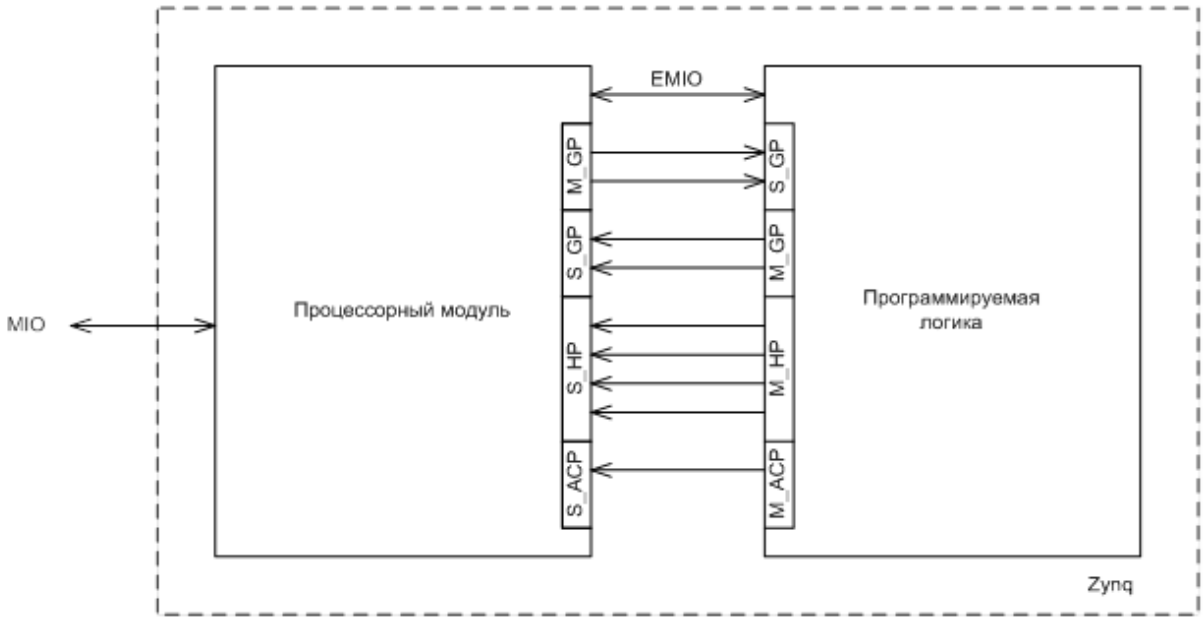


Рис. 4 — Структурная схема Zynq

Схема интерфейсов Zynq

**Буквы S и M у порта обозначают соответственно Slave и Master.*

Так как, в одном корпусе Zynq реализованы и процессорный модуль и программируемая логика, есть выводы, которые относятся к процессорному модулю и выводы, которые относятся к программируемой логике.

Порты

Порты MIO представляют собой многофункциональные порты ввода-вывода, непосредственно подключенные к выводам процессорной системы (Processing System, PS). Ключевые характеристики:

- **Количество:** 54 порта (в большинстве конфигураций)
- **Назначение:** подключение периферийных устройств PS к внешним выводам кристалла
- **Особенности:** мультиплексирование функций на одних и тех же физических выводах

MIO

Порты MIO подключены к выводам процессора. С помощью MIO могут быть подключены следующие периферийные устройства процессорного модуля:

- USB-контроллер – 2 шт
- Gigabit Ethernet контроллер – 2 шт
- SD/SDIO контроллер – 2 шт
- UART – 2 шт
- CAN – 2 шт
- I2C – 2 шт
- SPI – 2 шт
- GPIO. Все выводы можно использовать как выводы общего назначения

Так же, к MIO могут быть подключены следующие устройства памяти процессорного модуля:

- QSPI контроллер
- ONFI контроллер
- SRAM/NOR контроллер

Количество MIO портов равно 54 (за исключением некоторых микросхем в корпусе CLG225, там еще меньше). Поэтому все сразу включить не удастся. Для решения этой проблемы существует группа портов EMIO.

Выполнение

Практическая работы выполнялась в программе GNU Radio. GNU Radio - это открытая платформа для разработки программных решений в области SDR. Её ключевое преимущество — визуальное проектирование радиоэлектронных систем с помощью готовых блоков, что избавляет от необходимости программировать. Собранные проекты работают непосредственно с SDR-оборудованием, таким как Adalm-Pluto или LimeSDR.

Библиотека GNU Radio содержит обширный набор компонентов для цифровой обработки сигналов (ЦОС). Сами модули разработаны на C++ для обеспечения высокой производительности, а их соединение и управление проектом осуществляется с помощью Python. Разработку можно вести как программно, используя API, так и визуально — в графической среде GNU Radio Companion (GRC).

Создание схемы в GNU Radio

Блок Options (Параметры проекта)

Данный блок определяет основные настройки всего проекта. Наиболее важные параметры:

- **Output Language (Язык выходного кода)**: определяет язык генерации исполняемой программы (Python или C++)
- **Generate Options (Опции генерации)**: задаёт тип используемого интерфейса

Блок Variable (Переменная)

Служит для объявления переменных. Переменная имеет имя (ID) и значение. В проекте часто определяется:

```
Variable ID: samp_rate
Value: 2.4M
```

Блок QT GUI Range (Ползунок)

Создает регулируемый ползунок для динамического изменения параметров во время работы программы.

- **Default Value**: значение при запуске
- **Start/Stop**: диапазон значений
- **Step**: шаг изменения

Блок PlutoSDR Source (Источник)

Обеспечивает подключение к SDR-модулю ADALM-Pluto и управление его параметрами.
Основные параметры:

- **IIO context URI**: IP-адрес устройства
- **Sample Rate**: частота дискретизации АЦП
- **LO Frequency**: частота гетеродина
- **Buffer size**: размер буфера

Блок Low Pass Filter (ФНЧ)

Фильтр нижних частот для ограничения полосы пропускания сигнала.
Параметры:

- **Decimation**: коэффициент уменьшения частоты дискретизации
- **Cutoff Freq**: частота среза
- **Sample Rate**: исходная частота дискретизации

Блок QT GUI Frequency Sink (Спектр)

Обеспечивает визуализацию спектра сигнала в реальном времени.

Блок QT GUI Time Sink (Осциллограф)

Отображает сигнал во временной области.

Блок WBFM Receive (FM-демодулятор)

Выполняет демодуляцию широкополосного FM-сигнала.

Блок Audio Sink (Выход на звуковую карту)

Направляет аудиопоток на звуковую карту компьютера.

Сборка системы

Все блоки соединяются в графическом редакторе GRC, формируя законченную схему FM-приёмника. Программа позволяет в реальном времени:

– Принимать радиопередачи
– Наблюдать спектр и форму сигнала
– Регулировать частоту настройки

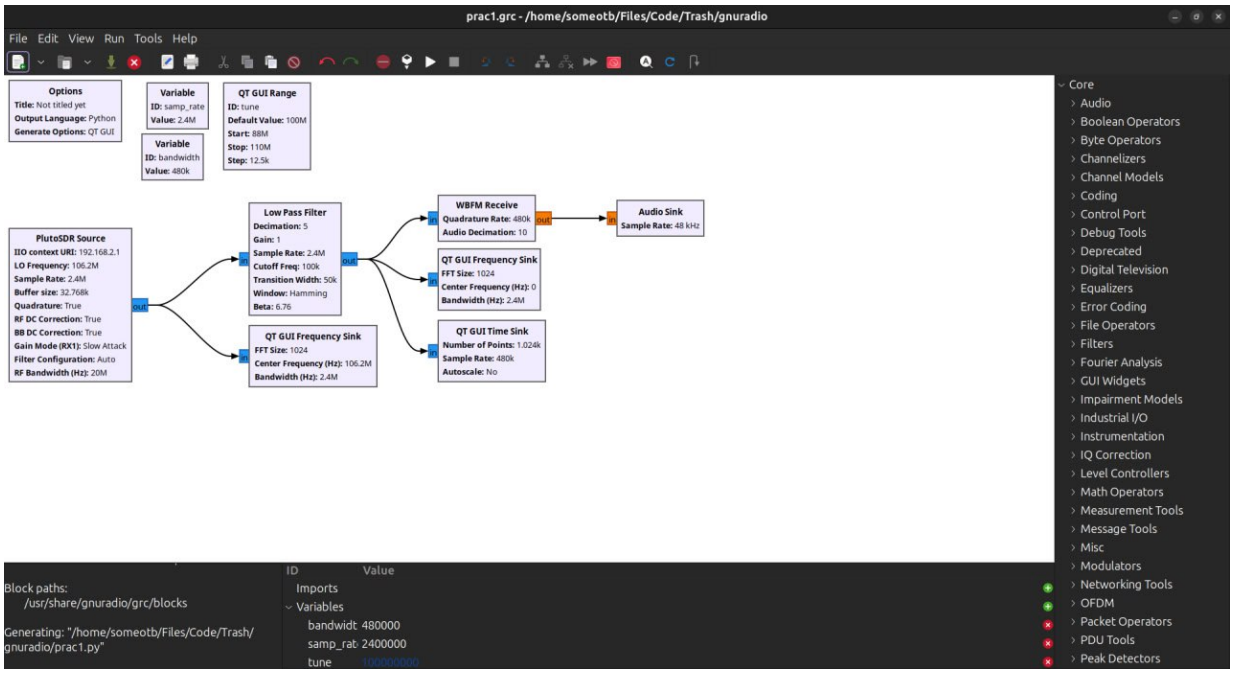


Рис. 5 — Собранная модель

Код собирается автоматически исходя из модели собранной из блоков в интерфейсе GNU Radio. И с помощью данной модели получилось поймать сигнал одной радио станции, которую можно было слушать, на других частотах звук был сильно искажен. На радио станции пока слушал, понравилась одна песня: Девочка в платьице белом - Мюзикола.

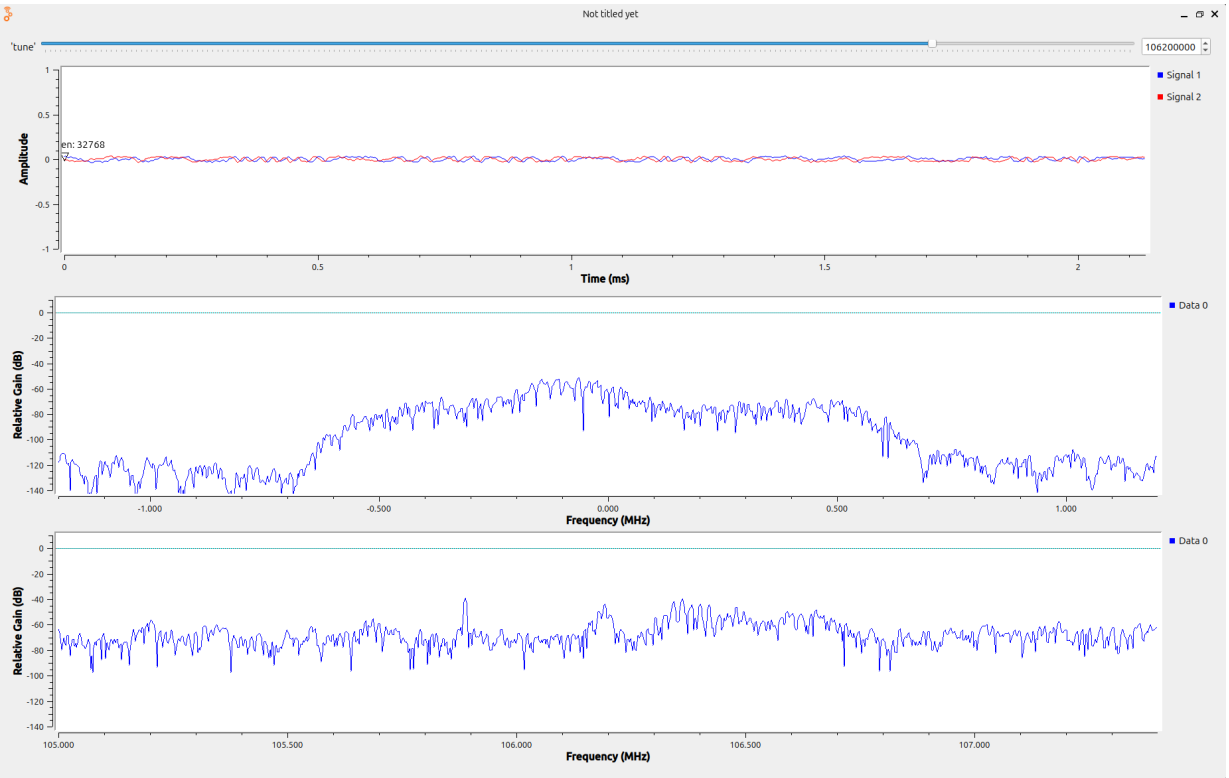


Рис. 6 — Результаты

Вывод

В результате работы были изучены принципы SDR, исследована архитектура ADALM Pluto и освоен инструмент GNU Radio. Разработана программа для приёма и воспроизведения передач в FM-диапазоне.

2 ЗНАКОМСТВО С БИБЛИОТЕКАМИ SOAPY SDR, LIBIIO ДЛЯ РАБОТЫ С ADALM PLUTO SDR. ИНИЦИАЛИЗАЦИЯ SDR-УСТРОЙСТВА. РАБОТА С БУФЕРОМ: ПОЛУЧЕНИЕ ЦИФРОВЫХ IQ-ОТСЧЕТОВ

Ссылка на GitHub

Цель практики:

Понять как работает приёмник. Написать на C++ программу для SDR. Передать свой сигнал и принять его, чтобы проверить как всё работает.

Краткие теоретические сведения

Начнем с описания сэмпла в рамках работы с Adalm Pluto.

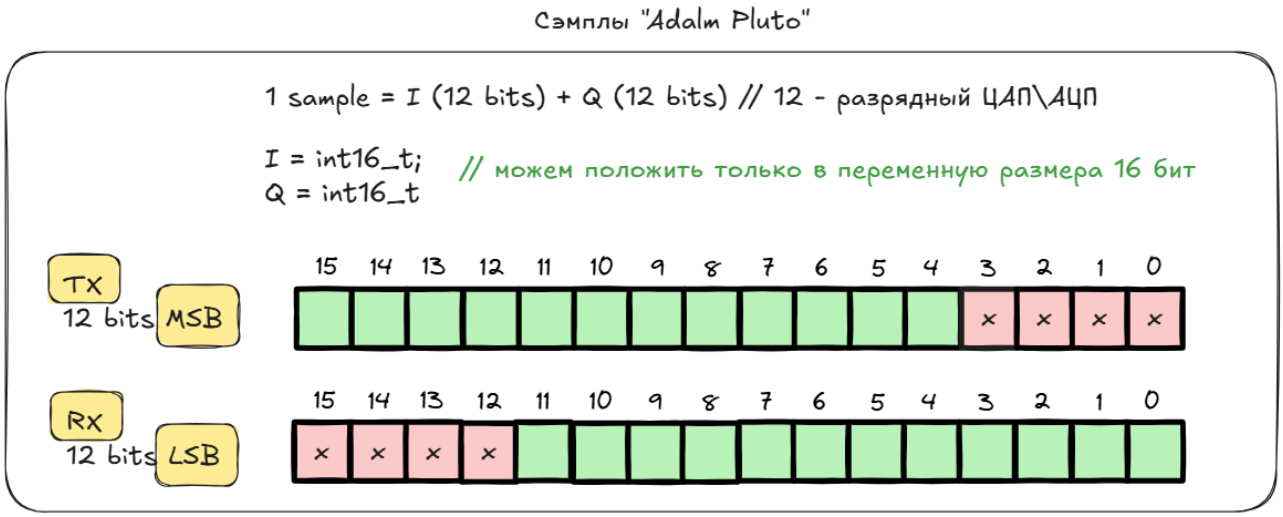


Рис. 7 — Сэмплы Adalm Pluto

Буфер, его структура, выделение памяти под буферы (для хранения сэмплов TX, RX):

Листинг 2.1 — Работа с буферами SDR

```
// Получение MTU
size_t rx_mtu = SoapySDRDevice_getStreamMTU(sdr, rxStream);
size_t tx_mtu = SoapySDRDevice_getStreamMTU(sdr, txStream);

// Выделениепамятиподбуферы
int16_t tx_buff[2*tx_mtu];
int16_t rx_buffer[2*rx_mtu];
```

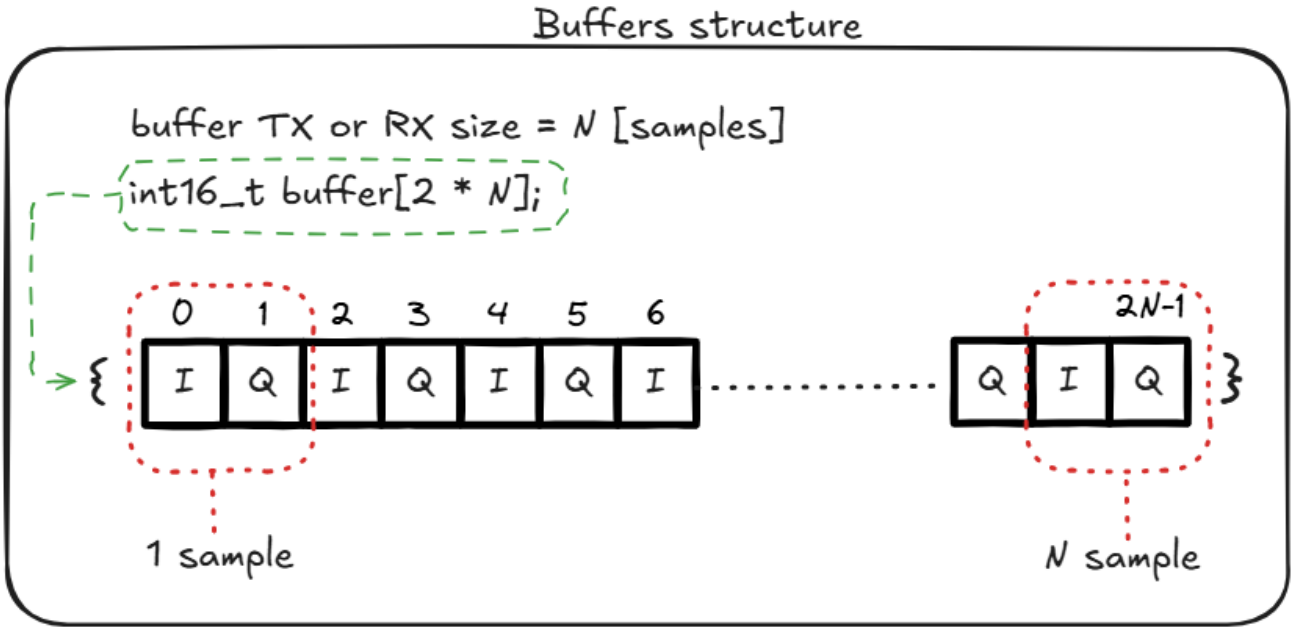


Рис. 8 — Структура буфера

Как происходит чтение сэмплов с SDR

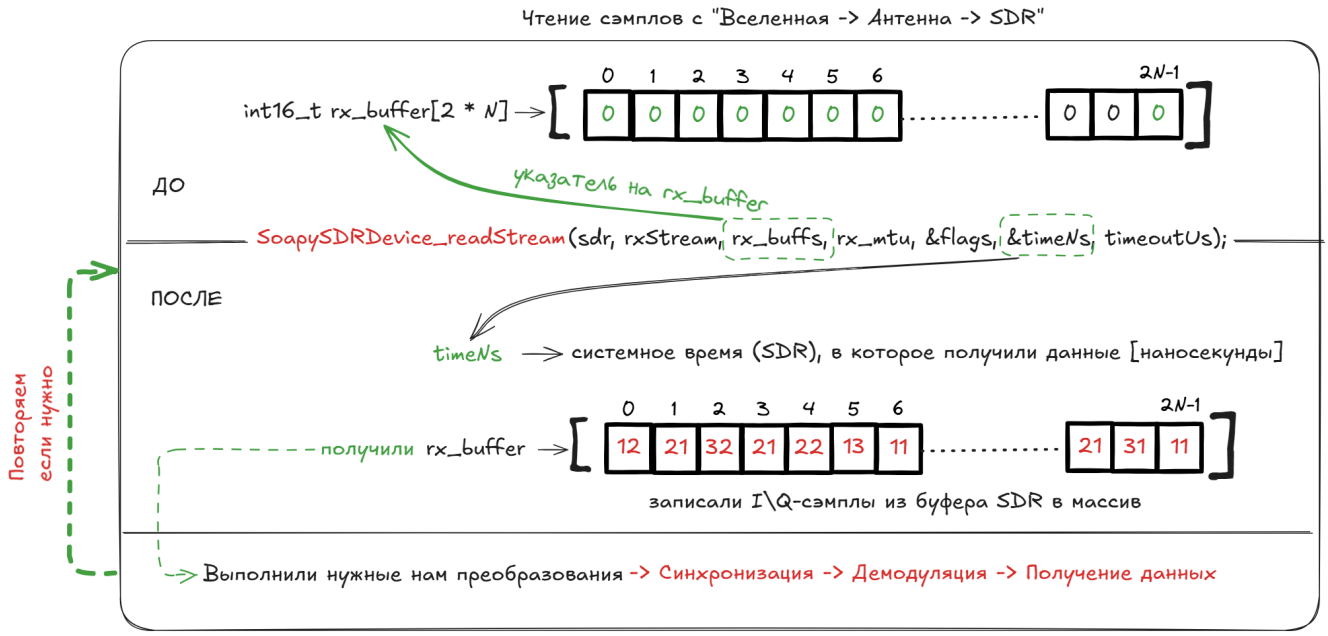


Рис. 9 — Чтение сэмплов

Листинг 2.2 — получения сэмплов с SDR

```
const long  timeoutUs = 400000;
long long last_time = 0;
size_t iteration_count = 10;

for (size_t buffers_read = 0; buffers_read < iteration_count; buffers_read++)
{
    void *rx_buffs[] = {rx_buffer};
```

```
    int flags;        // flags set by receive operation
    long long timeNs; //timestamp for receive buffer

    int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs, rx_mtu, &flags, &
        timeNs, timeoutUs);

    printf("Buffer: %lu - Samples: %i, Flags: %i, Time: %lli, TimeDiff: %lli\n",
        buffers_read, sr, flags, timeNs, timeNs - last_time);
}
```

Как происходит подготовка TX буфера и передача сэмплов на SDR

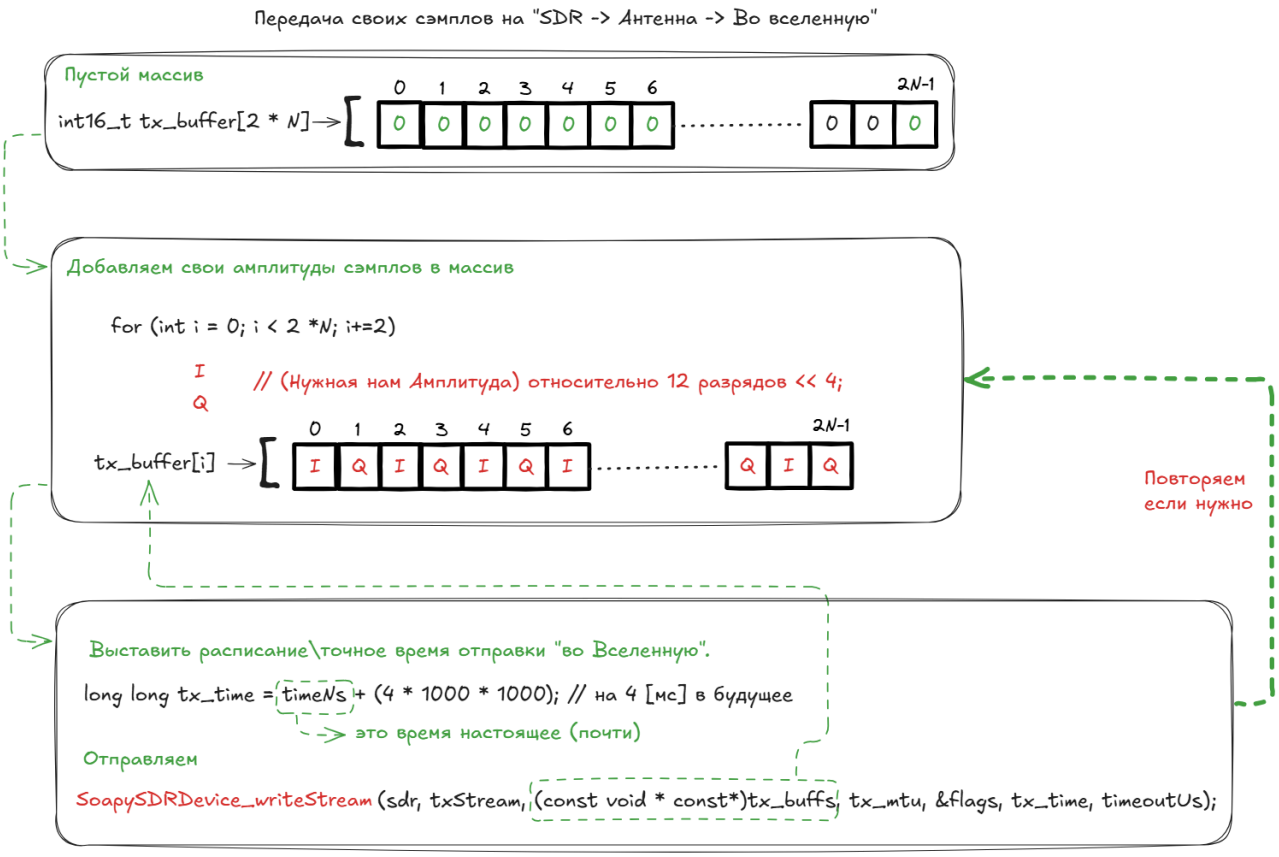


Рис. 10 — Передача сэмплов на SDR

Листинг 2.3 — Подготовка массива для передачи (TX buffer):

```
    for (int i = 2; i < 2 * tx_mtu; i+=2)
    {
        tx_buff[i] = 1500 << 4;   // I
        tx_buff[i+1] = 1500 << 4; // Q
    }
    //prepare fixed bytes in transmit buffer
    //we transmit a pattern of FFFF FFFF [TS_0]00 [TS_1]00 [TS_2]00 [TS_3]00 [TS_4]00
        [TS_5]00 [TS_6]00 [TS_7]00 FFFF FFFF
    //that is a flag (FFFF FFFF) followed by the 64 bit timestamp, split into 8 bytes
        and packed into the lsb of each of the DAC words.
```

```
    //DAC samples are left aligned 12-bits, so each byte is left shifted into place
    for(size_t i = 0; i < 2; i++)
    {
        tx_buff[0 + i] = 0xffff;
        // 8 x timestamp words
        tx_buff[10 + i] = 0xffff;
    }

    last_time = timeNs;

    long long tx_time = timeNs + (4 * 1000 * 1000); // на 4 мс\[\] вбудущее

    for(size_t i = 0; i < 8; i++)
    {
        uint8_t tx_time_byte = (tx_time >> (i * 8)) & 0xff;
        tx_buff[2 + i] = tx_time_byte << 4;
    }

    void *tx_buffs[] = {tx_buff};
    if( (buffers_read == 2) ){
        printf("buffers_read: %d\\n", buffers_read);
        flags = SOAPY_SDR_HAS_TIME;
        int st = SoapySDRDevice_writeStream(sdr, txStream, (const void * const*)
           tx_buffs, tx_mtu, &flags, tx_time, timeoutUs);
        if ((size_t)st != tx_mtu)
        {
            printf("TX Failed: %i\\n", st);
        }
    }
```

Одновременный прием и передача. Ниже представлена схема работы буферов RX и TX:

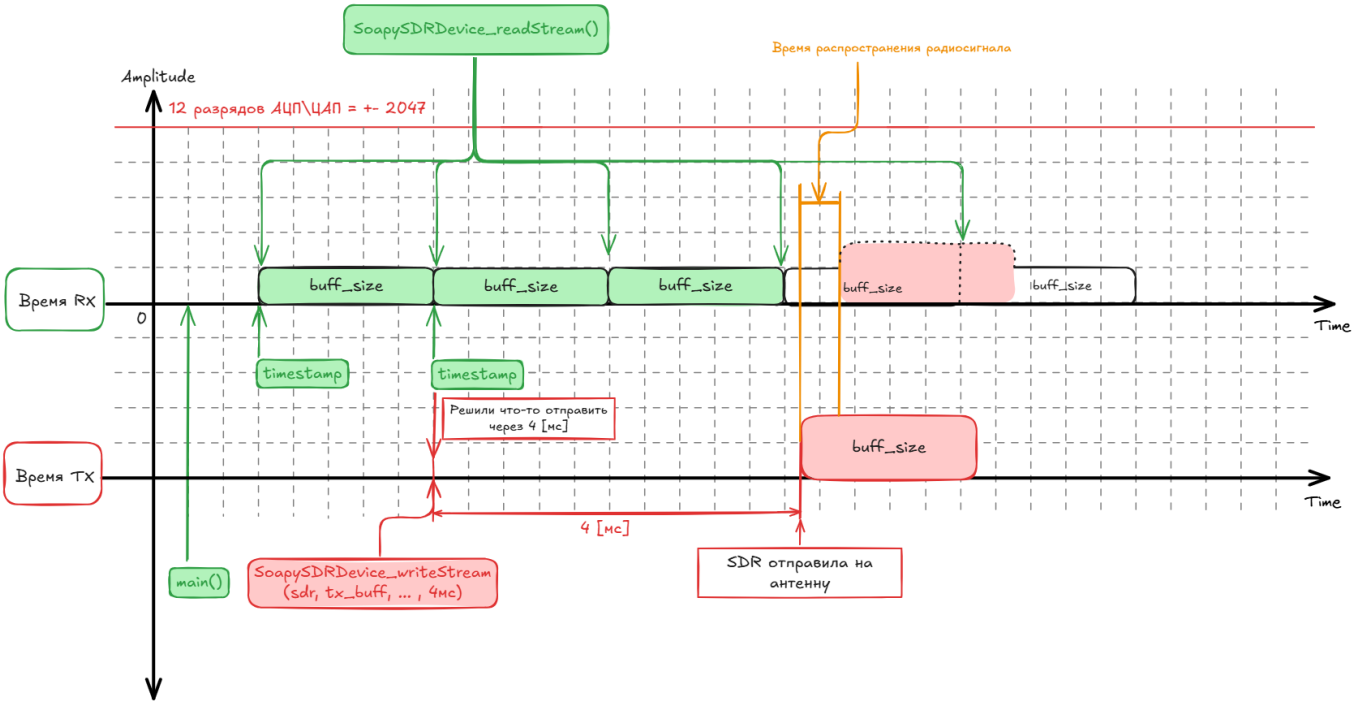


Рис. 11 — Одновременный прием и передача

Код, отражающий работу схемы:

Листинг 2.4 — Подготовка массива для передачи (TX buffer):

```
const long  timeoutUs = 400000;
long long last_time = 0;
size_t iteration_count = 10;

for (size_t buffers_read = 0; buffers_read < iteration_count; buffers_read++)
{
    void *rx_buffs[] = {rx_buffer};
    int flags;        // flags set by receive operation
    long long timeNs; //timestamp for receive buffer

    int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs, rx_mtu, &flags, &
       timeNs, timeoutUs);

    printf("Buffer: %lu - Samples: %i, Flags: %i, Time: %lli, TimeDiff: %lli\n",
       buffers_read, sr, flags, timeNs, timeNs - last_time);
    last_time = timeNs;

    long long tx_time = timeNs + (4 * 1000 * 1000); // на 4 мс[] вбудущее

    for(size_t i = 0; i < 8; i++)
    {
        uint8_t tx_time_byte = (tx_time >> (i * 8)) & 0xff;
        tx_buff[2 + i] = tx_time_byte << 4;
    }

```

```
    void *tx_buffs[] = {tx_buff};
    if( (buffers_read == 2) ){
        printf("buffers_read: %d\n", buffers_read);
        flags = SOAPY_SDR_HAS_TIME;
        int st = SoapySDRDevice_writeStream(sdr, txStream, (const void * const*)
           tx_buffs, tx_mtu, &flags, tx_time, timeoutUs);
        if ((size_t)st != tx_mtu)
        {
            printf("TX Failed: %i\n", st);
        }
    }

}
```

Выполнение

Сперва необходимо установить все необходимые библиотеки

Листинг 2.5 — Установка SoapySDR

```
sudo apt-get install python3-pip python3-setuptools
sudo apt-get install cmake g++ libpython3-dev python3-numpy swig python3-matplotlib

git clone --branch soapy-sdr-0.8.1 https://github.com/TelecomDep/SoapySDR.git

cd SoapySDR
mkdir build && cd build

cmake ../

make -j\`nproc\` # nproc - количествопотоков , например make -j16
sudo make install
sudo ldconfig
```

Листинг 2.6 — Установка Libiio

```
sudo apt-get install libxml2 libxml2-dev bison flex libcdk5-dev cmake
sudo apt-get install libusb-1.0-0-dev libaio-dev pkg-config
sudo apt install libavahi-common-dev libavahi-client-dev

git clone --branch v0.24 https://github.com/TelecomDep/libiio.git

cd libiio
mkdir build && cd build
cmake ../
make -j`nproc` # nproc - количествопотоков , например make -j16
sudo make install
```

Листинг 2.7 — Установка LibAD9361

```
git clone --branch v0.3 https://github.com/TelecomDep/libad9361-iio.git
cd libad9361-iio

mkdir build && cd build

cmake ../

make -j\`nproc\` # nproc - количествопотоков , например make -j16
sudo make install
sudo ldconfig
```

Листинг 2.8 — Установка SoapyPlutoSDR

```
git clone --branch sdr_gadget_timestamping https://github.com/TelecomDep/
    SoapyPlutoSDR.git
cd SoapyPlutoSDR

mkdir build && cd build

cmake ../

make -j\`nproc\` # nproc - количествопотоков , например make -j16
sudo make install
sudo ldconfig
```

Соответственно подключить их в самом коде

Листинг 2.9 — Подключение бибилиотек

```
#include <SoapySDR/Device.h>   // Инициализацияустройства
#include <SoapySDR/Formats.h>  // Типыданных , используемыхдлязаписисэмплов
#include <stdio.h>             //printf
#include <stdlib.h>            //free
#include <stdint.h>
#include <complex.h>
```

Листинг 2.10 — Аргументы(в рамках библиотки SoapySDR имеют формат ключ:значение)

```
SoapySDRKwargs args = {};
SoapySDRKwargs_set(&args, "driver", "plutosdr");
if (1) {
    SoapySDRKwargs_set(&args, "uri", "usb:");
} else {
    SoapySDRKwargs_set(&args, "uri", "ip:192.168.2.1");
}
SoapySDRKwargs_set(&args, "direct", "1");
SoapySDRKwargs_set(&args, "timestamp_every", "1920");
SoapySDRKwargs_set(&args, "loopback", "0");
SoapySDRDevice \*sdr = SoapySDRDevice_make(&args);
SoapySDRKwargs_clear(&args);
```

Мы подключаем SDR к нашему ПК или ноутбуку по USB2.0, в момент запуска программы SDR начинает выполнять запрограммированные действия, в нашем случае мы отправляем десять буферов, которые записываются в бинарный файл с расширением .pcm(Pulse-Code Modulation - импульсно-кодовая модуляция), который мы далее визуализируем с помощью python, и его библиотек matplotlib, numpy

Листинг 2.11 — Код для визуализации полученных данных с SDR

```python
import matplotlib.pyplot as plt
import numpy as np

rx = np.fromfile("/home/plutoSDR/sdr/pluto/dev/rx.pcm", dtype=np.int16)

samples = []
for x in range(0, len(rx), 2):
    samples.append(rx[x] + 1j * rx[x+1])

samples = np.array(samples)

ampl = np.abs(samples)
phase = np.angle(samples)
time = np.arange(len(samples))

plt.figure(figsize=(10, 8))

plt.subplot(3, 1, 1)
plt.plot(time, ampl)
plt.title("Amplitude")
plt.ylabel("Amplitude")
plt.grid(True)

plt.subplot(3, 1, 2)
plt.plot(time, phase)
plt.title("Phase")
plt.ylabel("Phase (rad)")
plt.grid(True)

plt.subplot(3, 1, 3)
plt.plot(time, np.real(samples), label='I (Real)')
plt.plot(time, np.imag(samples), label='Q (Imag)')
plt.title("I and Q components")
plt.ylabel("Value")
plt.xlabel("Time (samples)")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

Полученные Результаты приведены на графике, через subplot, на первом subplot показана амплитуда полученного сигнала, на втором subplot показана фаза полученного сигнала, на третьем I - реальная и Q - мнимая части.

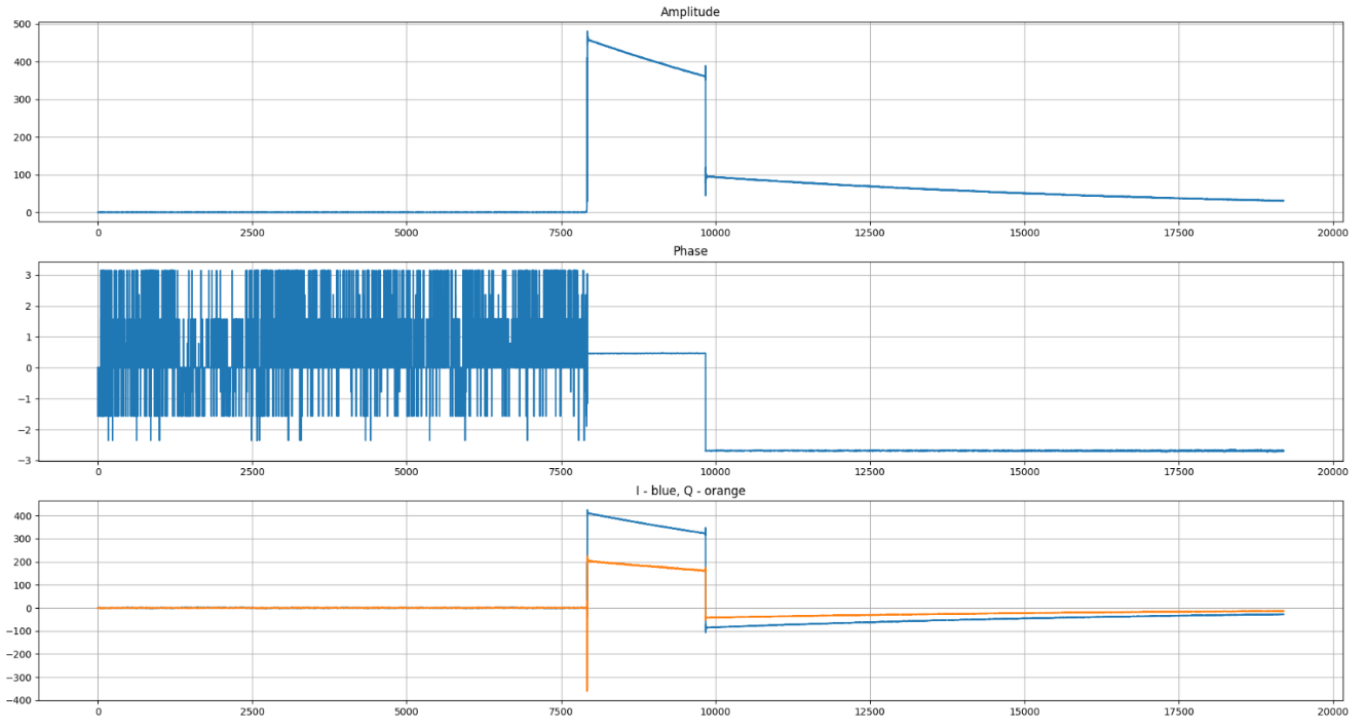


Рис. 12 — Результаты, которые мы получили с SDR

Вывод

В ходе работы я познакомился с минимальной схемой работы Pluto SDR, установил и подключил нужные библиотеки(LibAD9361, Libiio, SoapySDR, SoapyPlutoSDR), отправил сэмплы и получил их, вывел характеристики полученного сигнала на график.

3 РАБОТА С БИБЛИОТЕКАМИ SOAPY SDR, LIBIIO. ФОРМИРОВАНИЕ И ПЕРЕДАЧА С SDR СИГНАЛОВ ПРОИЗВОЛЬНОЙ ФОРМЫ

Ссылка на GitHub

Цель практики:

Лучше разобраться с библиотеками, сформировать сэмплы и отправить их, визулизировать полученые данные.

Краткие теоретические сведения

Вспомним структуру отправляемого буфера

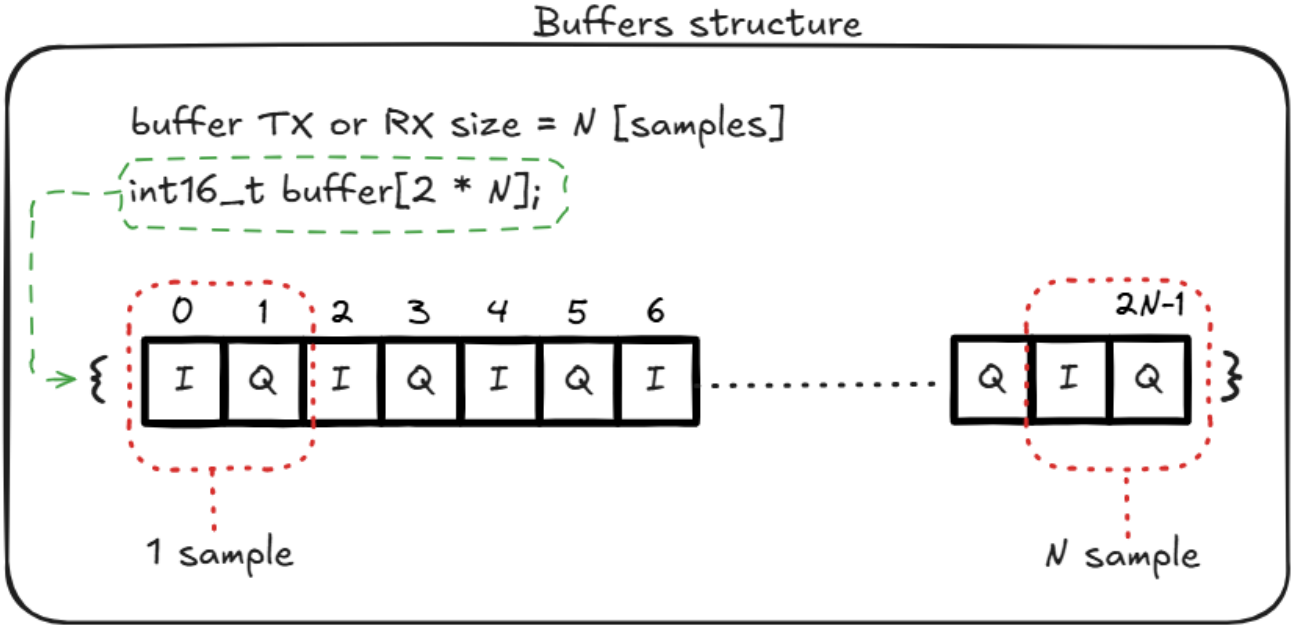


Рис. 13 — Структура буфера

Структура сэмплов

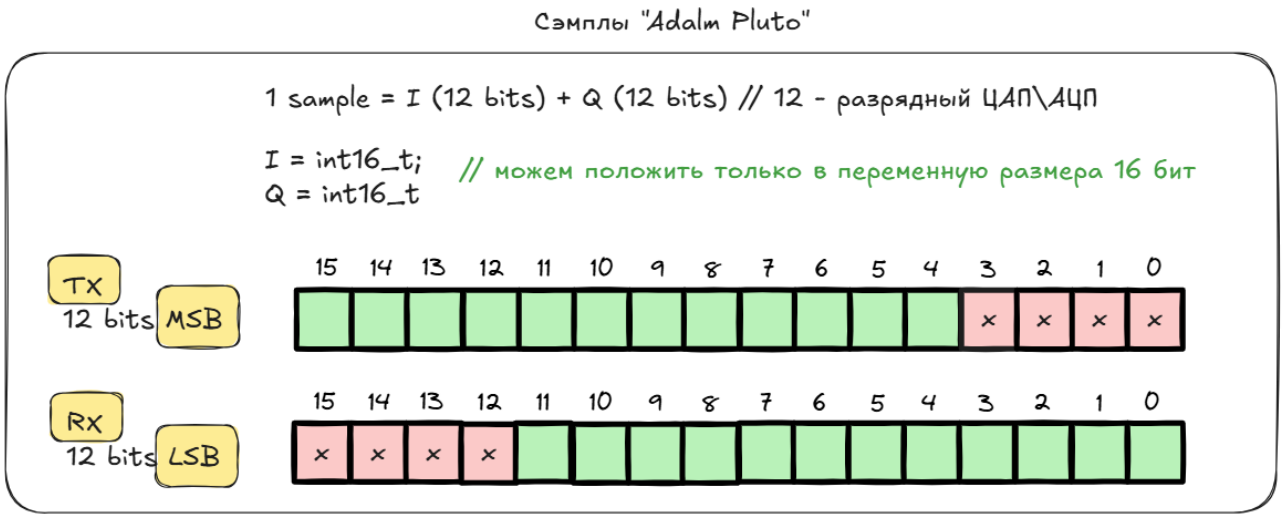


Рис. 14 — Структура сэмплов

Как читаются сэмплы

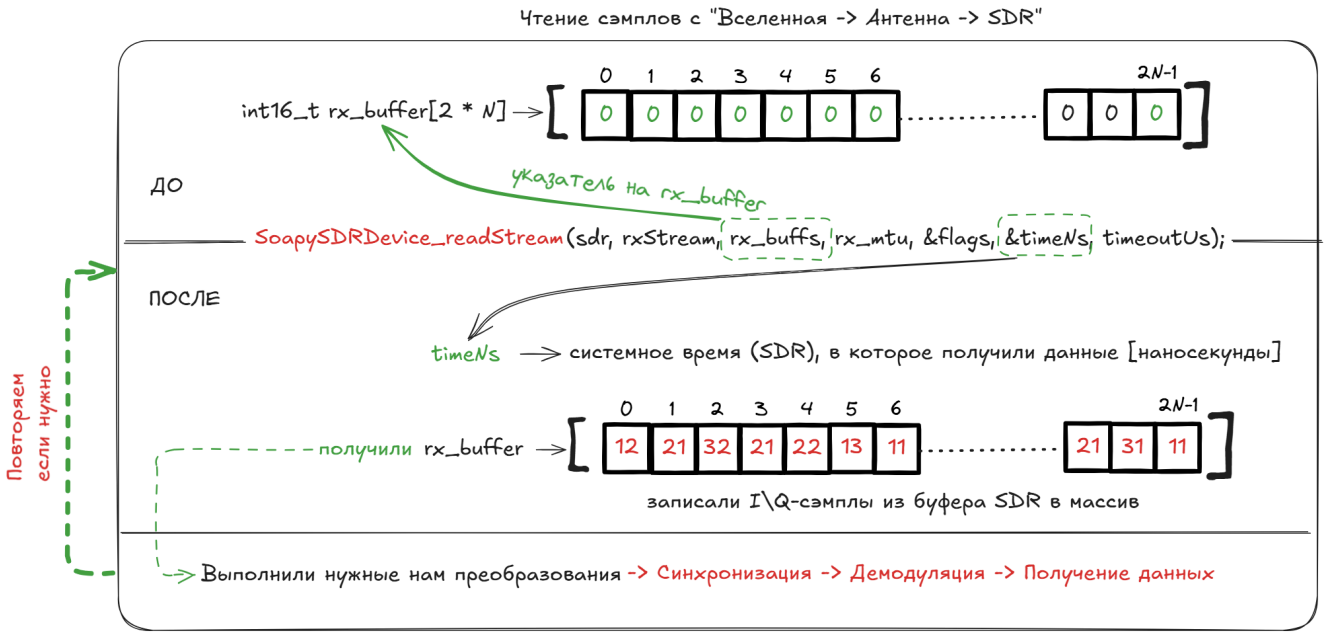


Рис. 15 — Чтение сэмплов

Выполнение

Сперва я заполнил буферы значениями 1500 « 4 ($1500 * 2^4 = 24000$) Передавал их на SDR и смотрел что получается. Визуализировать удобно с помощью кода на python

Листинг 3.1 — Код для визуализации полученных данных с SDR

```python
import matplotlib.pyplot as plt
import numpy as np

rx = np.fromfile("/home/plutoSDR/sdr/pluto/dev/rx.pcm", dtype=np.int16)
```

```
5
6  samples = []
7  for x in range(0, len(rx), 2):
8      samples.append(rx[x] + 1j * rx[x+1])
9
10 samples = np.array(samples)
11
12 ampl = np.abs(samples)
13 phase = np.angle(samples)
14 time = np.arange(len(samples))
15
16 plt.figure(figsize=(10, 8))
17
18 plt.subplot(3, 1, 1)
19 plt.plot(time, ampl)
20 plt.title("Amplitude")
21 plt.ylabel("Amplitude")
22 plt.grid(True)
23
24 plt.subplot(3, 1, 2)
25 plt.plot(time, phase)
26 plt.title("Phase")
27 plt.ylabel("Phase (rad)")
28 plt.grid(True)
29
30 plt.subplot(3, 1, 3)
31 plt.plot(time, np.real(samples), label='I (Real)')
32 plt.plot(time, np.imag(samples), label='Q (Imag)')
33 plt.title("I and Q components")
34 plt.ylabel("Value")
35 plt.xlabel("Time (samples)")
36 plt.legend()
37 plt.grid(True)
38
39 plt.tight_layout()
40 plt.show()
```

На графике получаем, амплитуду, фазу и I,Q части

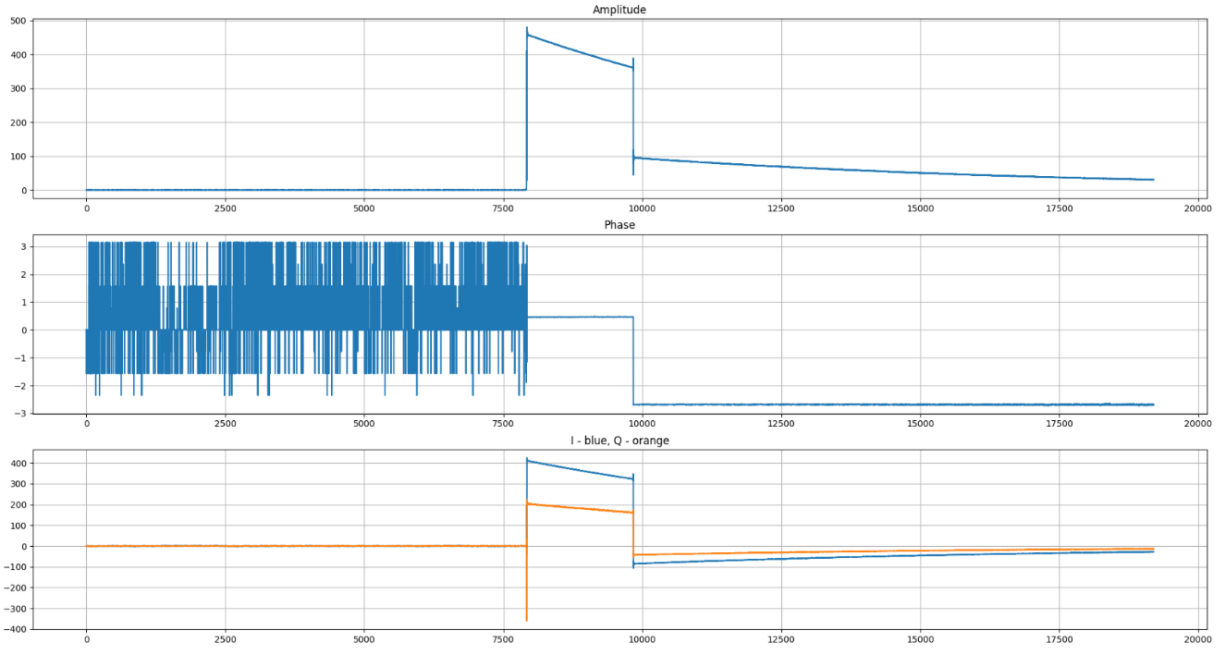


Рис. 16 — Графики

Вывод

Самостоятельно записали произвольные данные в буферы, который потом отправили на SDR, получили их, вывели на графики, с помощью python кода, наглядно показано, что получаем на выходе после всех этапов отправки данных.

4 ИМИТАЦИЯ АНАЛОГОВОЙ ПЕРЕДАЧИ ЗВУКА И ЕГО ПРИЕМ С ИСПОЛЬЗОВАНИЕМ SDR. АНАЛИЗ ВЛИЯНИЯ ЧУВСТВИТЕЛЬНОСТИ ПРИЕМНИКА И УСИЛЕНИЯ ПЕРЕДАТЧИКА НА КАЧЕСТВО ПРИНЯТЫХ ОТСЧЕТОВ СИГНАЛА (СЕМПЛОВ)

Ссылка на GitHub

Цель практики:

Попробовать отправить свою аудио запись, получить ее и посмотреть что поменялось. Попробовать получить эффект сложения сигналов, путем скрещения антен двух Pluto SDR.

Краткие теоретические сведения

Сперва что такое MP3, PCM?

PCM (Pulse Code Modulation) — это линейное, несжатое представление звука. Сигнал записывается как последовательность отсчётов амплитуды с фиксированной частотой дискретизации и глубиной квантования. PCM хранит аудио без сжатия, что делает его максимально точным, но объёмным.

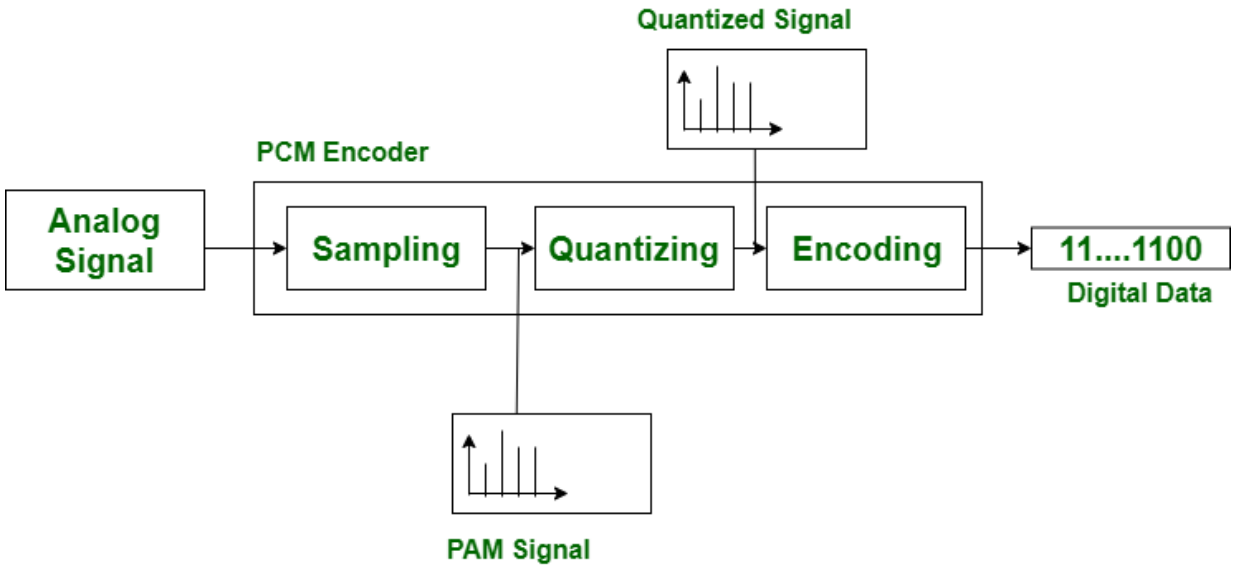


Рис. 17 — Графики

MP3 (MPEG-1 Audio Layer III) — это формат сжатия аудио с потерями, основанный на психоакустической модели. Он удаляет компоненты звука, менее заметные человеческому слуху, и применяет дополнительные методы кодирования, что существенно уменьшает размер файла.

Для конвертации между этими форматами можно использовать библиотеку pydub в Python. А для конвертации MP3 в WAV можно использовать ffmpeg.

Листинг 4.1 — Код на C++ для чтения PCM файла

```
int16_t *read_pcm(const char *filename, size_t *sample_count)
{
    FILE *file = fopen(filename, "rb");

    fseek(file, 0, SEEK_END);
    long file_size = ftell(file);
    fseek(file, 0, SEEK_SET);
    printf("file_size = %ld\n", file_size);

    *sample_count = file_size / sizeof(int16_t);
    int16_t *samples = (int16_t *)malloc(file_size);
    size_t sf = fread(samples, sizeof(int16_t), *sample_count, file);

    if (sf == 0){
        printf("file %s empty!", filename);
    }

    fclose(file);

    return samples;
}
```

Выполнение

Для начала, я написал код для конвертации mp3 файла в pcm формат с помощью pydub.

Листинг 4.2 — Код на python для конвертации mp3 в pcm

```
import numpy as np
import librosa
from pydub import AudioSegment

def mp3_to_pcm(src_path, dst_path, sample_rate=44100):
    audio_data, sr = librosa.load(src_path, sr=sample_rate, mono=False)
    if audio_data.ndim > 1:
        audio_data = audio_data.T.flatten()
    pcm_array = np.int16(audio_data * 32767)
    pcm_array.tofile(dst_path)

def pcm_to_mp3(pcm_path, output_path, channels=2, sample_rate=44100, bitrate="160k"):
    raw_audio = np.fromfile(pcm_path, dtype=np.int16)
    sound = AudioSegment(
        data=raw_audio.tobytes(),
        sample_width=2,
```

```
        frame_rate=sample_rate,
        channels=channels
    )
    sound.export(output_path, format="mp3", bitrate=bitrate)

print("""PCM->MP3: 0
MP3->PCM: 1""")
ans = int(input("Выберите режим: "))
if ans == 1:
    mp3_to_pcm("../1.mp3", "../1.pcm")
else:
    pcm_to_mp3("../2.pcm", "../2.mp3")
```

Он на выбор преобразует mp3 файл в pcm или наоборот. Используем его перед началом экспериментов. В main.cpp я добавил чтение pcm файла и заполнение буфера прочитанными значениями.

Листинг 4.3 — Заполнения буфера данными из pcm

```
size_t iteration_count = 10;
long long last_time = 0;

FILE* file = fopen("../rx.pcm", "wb");
FILE* file1 = fopen("../tx.pcm", "wb");

for (size_t buffers_read = 0; buffers_read < iteration_count; buffers_read++)
    {
        void *rx_buffs[] = {rx_buffer};
        int flags;        // flags set by receive operation
        long long timeNs; //timestamp for receive buffer
        long timeoutUs = 1000000;
        int sr = SoapySDRDevice_readStream(sdr, rxStream, rx_buffs, rx_mtu, &flags, &
            timeNs, timeoutUs);

        printf("Buffer: %lu - Samples: %i, Flags: %i, Time: %lli, TimeDiff: %lli\n",
            buffers_read, sr, flags, timeNs, timeNs - last_time);
        last_time = timeNs;

        long long tx_time = timeNs + (4 * 1000 * 1000); // на 4 мс[] вбудущее

        for(size_t i = 0; i < 8; i++)
        {
            uint8_t tx_time_byte = (tx_time >> (i * 8)) & 0xff;
            tx_buff[2 + i] = tx_time_byte << 4;
        }

        void *tx_buffs[] = {tx_buff};
        if( (buffers_read==2) ){
            flags = SOAPY_SDR_HAS_TIME;
            int st = SoapySDRDevice_writeStream(sdr, txStream, (const void * const*)
                tx_buffs, tx_mtu, &flags, tx_time, timeoutUs);
```

```
            if ((size_t)st != tx_mtu)
            {
                printf("TX Failed: %i\n", st);
            }
        }

        fwrite(rx_buffer, 2 * rx_mtu * sizeof(int16_t), 1, file);
        fwrite(tx_buffs, 2 * rx_mtu * sizeof(int16_t), 1, file1);
    }

fclose(file);
```

Не забываем закрыть файл через fclose, и получается, что мы записали и отправили наш pcm файл через Pluto SDR, rx_buffer в file(rx.pcm) и tx_buffer в file1(tx.pcm). С помощью нашего кода на python мы можем конвертировать pcm обратно в mp3, чтобы прослушать, полученные mp3 получились искаженными. Если скрестить две Pluto SDR, одновременно начать отправку сэплов, то получится эффект наложения сигналов. У нас с одногруппником получилось так, что одновременно были слышны 2 песни.

Вывод

Я научился переводить файлы из mp3 в pcm и обратно, записывать их в буфер Pluto SDR, отправлять их и получать, полученные файлы были конвертированы обратно в mp3 для того чтобы прослушать, что получилось передать.

5 ДИСКРЕТНАЯ СВЕРТКА. РЕАЛИЗАЦИЯ ПРИЕМА И ПЕРЕДАЧИ BPSK СИГНАЛОВ

Ссылка на GitHub

Цель практики:

Програмно реализовать дискретную свертку, используя язык программировани C++, передать полученные после свертки биты, посмотреть, что мы получим на выходе

Краткие теоретические сведения

Дискретная свертка — это математическая операция над двумя дискретными последовательностями (сигналами), определяемая следующим образом:

Пусть даны две последовательности x[n] и h[n], где n $n \in Z$. Их свертка y[n] вычисляется по формуле:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n-k]$$

Для последовательностей конечной длины L и M, где x[n] определена при $0 \leq n \leq L-1$, а h[n] — при $0 \leq n \leq M-1$, результирующая последовательность $y[n]$ будет иметь длину $N = L + M - 1$ и формула принимает вид:

$$y[n] = \sum_{k=\max(0,n-M+1)}^{\min(n,L-1)} x[k] \cdot h[n-k], \quad \text{для } 0 \leq n \leq N-1$$

Свойства дискретной свертки:

- **Коммутативность:** $x * h = h * x$
- **Ассоциативность:** $(x * h) * g = x * (h * g)$
- **Дистрибутивность:** $x * (h + g) = (x * h) + (x * g)$

Передавать мы будем биты после свертки. Перед этим биты нужно удлинить, за это будет отвечать функция upsample, которая из одного бита делает напирмер 10. То есть у нас было [1011] битов, а стало(если upsample до 5) [10000,00000,10000,10000], далее эти биты идут в функцию под названием filter, которая выполняет свертку с еденичным импольсом после которой у нас получается такой результат [11111,00000,11111,11111]. Получается, что мы продлеваем наш сигнал.

BPSK и QPSK

BPSK (Binary Phase Shift Keying) — это цифровая модуляция, при которой двоичные символы 0 и 1 передаются изменением фазы несущего сигнала на 0 и π радиан соответственно. Сигнал BPSK можно записать как:

$$s_{\text{BPSK}}(t) = A\cos(2\pi f_c t + \phi), \quad \phi = \begin{cases} 0, & \text{для символа } 1, \\ \pi, & \text{для символа } 0. \end{cases}$$

QPSK (Quadrature Phase Shift Keying) — это модуляция, передающая два бита за символ с использованием четырёх фазовых состояний, разнесённых на $\frac{\pi}{2}$ радиан. Сигнал QPSK имеет вид:

$$s_{\text{QPSK}}(t) = A\cos\left(2\pi f_c t + \frac{(2k+1)\pi}{4}\right), \quad k = 0, 1, 2, 3,$$

что соответствует фазам $\frac{\pi}{4}, \frac{3\pi}{4}, \frac{5\pi}{4}, \frac{7\pi}{4}$.

BPSK обеспечивает более высокую помехоустойчивость, тогда как QPSK удваивает спектральную эффективность по сравнению с BPSK при том же энергетическом расходе на бит.

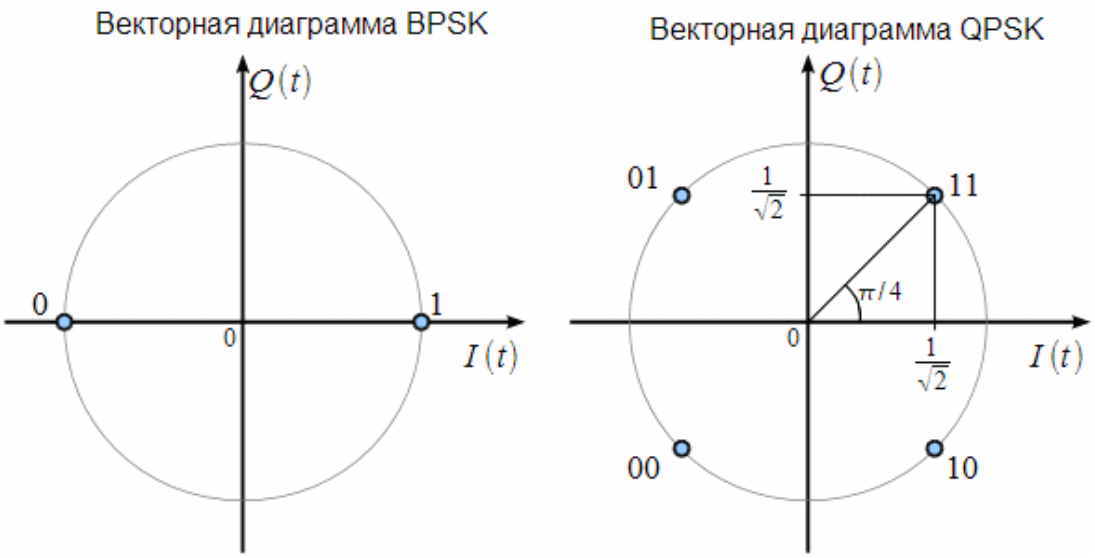


Рис. 18 — Векторная диаграмма BPSK, QPSK

На осцилографе QPSK сигнала, должно быть видно как меняется фаза

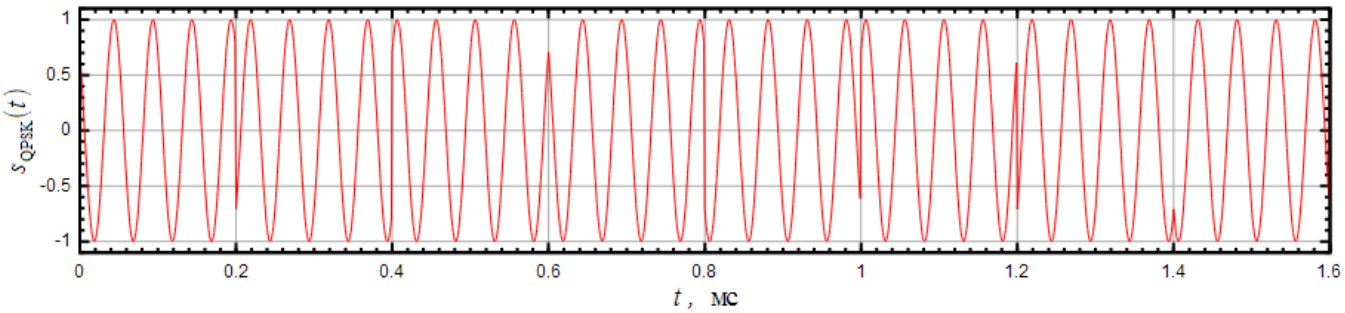


Рис. 19 — Осциллограмма QPSK сигнала

Структурная схема QPSK модулятора будет выглядеть так:

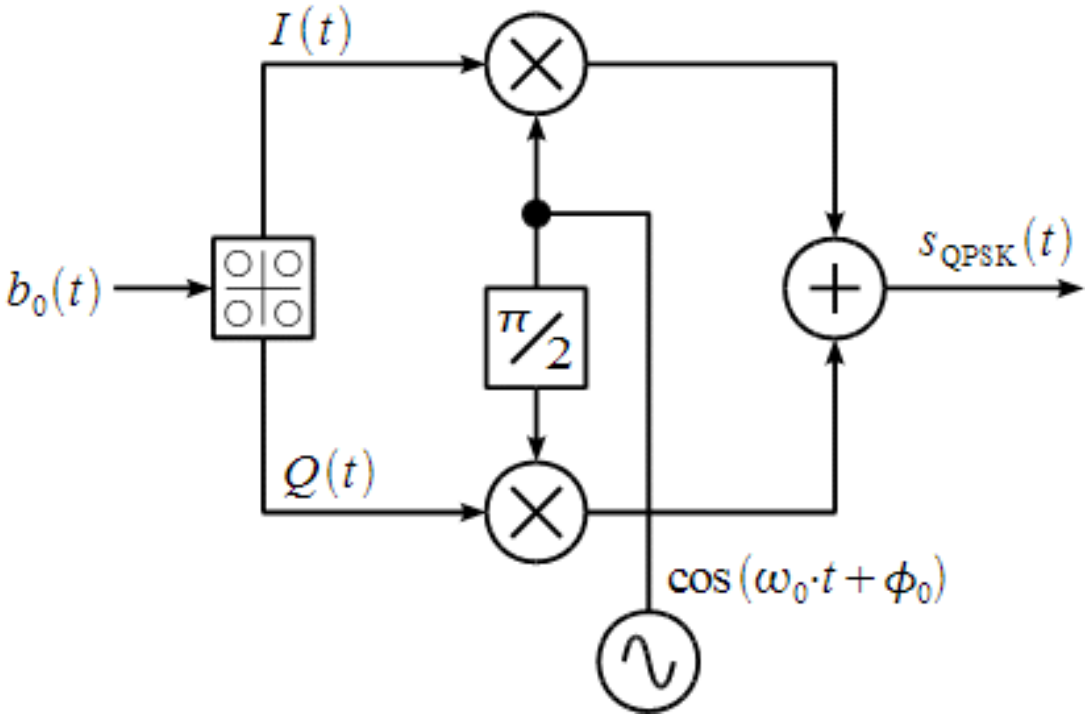


Рис. 20 — Структурная схема QPSK модулятора

Выполнение

Вместо BPSK, будем использовать QPSK. Напишем код для перевода в QPSK символы

Листинг 5.1 — map для QPSK

```
// QPSK символы
const complex<double> i0 (0, 0);
const complex<double> i1 (1, 1);
const complex<double> i2 (-1, 1);
const complex<double> i3 (-1, -1);
const complex<double> i4 (1, -1);
```

```
7
8  static map<string, complex<double>> qpsk_map = {
9      {"00", i1},
10     {"01", i2},
11     {"11", i3},
12     {"10", i4}
13 };
```

Функция для UpSampler, которая удлиняет сигнал, реализована с помощью простых циклов

Листинг 5.2 — UpSampler

```
1 void UpSampler(complex<double> *symbols, int len_s, complex<double> *symbols_ups, int
      L){
2     for (size_t i = 0; i < (size_t)len_s*(size_t)L; i++) symbols_ups[i] = i0;
3     for (size_t i = 0; i < (size_t)len_s; i++) symbols_ups[i*L] = symbols[i];
4 }
```

Функция для свертки, называется в моем коде filter, реализует процесс свертки массива уже после upsample с еденичным импульсом с заданной нами длиной

Листинг 5.3 — filter

```
1 void filter(complex<double> *symbols_ups, int len_symbols_ups, complex<double> *
     impulse, int L) {
2 vector<complex<double>> sum(len_symbols_ups, 0);
3 for (size_t i = 0; i < (size_t)len_symbols_ups; i++) {
4     for (size_t j = 0; j < (size_t)L && (int)(i-j) >= 0; j++) {
5         sum[i] += impulse[j] * symbols_ups[i-j];
6     }
7 }
8 memcpy(symbols_ups, sum.data(), len_symbols_ups * sizeof(complex<double>));
9 }
```

Функция Mapper, которая формирует из 2 битов один символ, который далее соотносится к соотвующему комплексному значению в нашем qpsk_map.

Листинг 5.4 — Mapper

```
1 void Mapper(int16_t *bits, int len_b, complex<double> *symbols){
2     string pair_bits;
3     for (size_t i = 0; i < (size_t)len_b; i += 2){
4         pair_bits = to_string(bits[i]) + to_string(bits[i+1]);
5         symbols[i/2] = qpsk_map[pair_bits];
6     }
7 }
```

Для практики я рандомно формирую 20 битов от 0 до 1, затем пропускаю их через Mapper для того чтобы сформировать из битов символы по 2 бита каждый, далее пропускаю эти символы через UpSampler чтобы ”продлить”сигнал, затем подаю уже удлинненые символы на filter, который делает свертку с еденичным импульсом

Листинг 5.5 — Как это устроено в коде

```
int n = 20;
int16_t *bits = (int16_t*)malloc(n * sizeof(int16_t));
for (int i = 0; i < n; i++) bits[i] = rand() % 2;

int len_symbols = n / 2;
complex<double> *symbols = (complex<double>*)malloc(len_symbols * sizeof(complex<
    double>));
int L = 10;
int len_symbols_ups = len_symbols * L;
complex<double> *symbols_ups = (complex<double>*)malloc(len_symbols_ups * sizeof(
    complex<double>));
complex<double> impulse[L];
for (int i = 0; i < L; i++) impulse[i] = 1.0;

Mapper(bits, n, symbols);
UpSampler(symbols, len_symbols, symbols_ups, L);
filter(symbols_ups, len_symbols_ups, impulse, L);
// Show_ArrayСимволы("", symbols_ups, len_symbols_ups);

int16_t *tx_samples = (int16_t*)malloc(2 * config.tx_mtu * sizeof(int16_t));
for (size_t i = 0; i < (size_t)config.tx_mtu; i++) {
    tx_samples[2*i] = (int16_t)((real(symbols_ups[i])) * 1000) << 4;
    tx_samples[2*i + 1] = (int16_t)((imag(symbols_ups[i])) * 1000) << 4;
}

// memcpy(config.tx_buff, tx_samples, 2 * len_symbols_ups * sizeof(int16_t));
fwrite(tx_samples, sizeof(int16_t), 2 * len_symbols_ups, tx);
```

Так же для того чтобы увидеть наши биты нам нужен код на python, который будет визулизировать все, что мы получили

Листинг 5.6 — Визуализация полученных битов

```
import matplotlib.pyplot as plt
import numpy as np

print("""Choose 0 - Tx
Choose 1 - Rx""")

answer = int(input())

if answer == 1:
    nx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np
        .int16)
else:
    nx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/tx.pcm", dtype=np
        .int16)

```

```
rx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
    int16)
tx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/tx.pcm", dtype=np.
    int16)
print(f"RX: {rx}\nTX: {tx}")

samples = []

for x in range(0, len(nx), 2):
    samples.append(nx[x] + 1j * nx[x+1])

ampl = np.abs(samples)
phase = np.angle(samples)
time = np.arange(len(samples))

# plt.subplot(3,1,1)
# plt.plot(time, ampl)
# plt.grid(True)
# plt.tight_layout()
# plt.title("Amplitude")

# plt.subplot(3,1,2)
# plt.plot(time, phase)
# plt.grid(True)
# plt.tight_layout()
# plt.title("Phase")


# plt.subplot(3,1,3)
plt.plot(time, nx[0::2])
# plt.plot(time, nx[1::2])
plt.grid(True)
plt.tight_layout()
plt.title("Real and Imag parts")
plt.show()
```

По итогу мы можем считатать наши pcm файлы, в которых содержатся наши сгенерированные биты от 0 до 1, можем вывести их амплитуду, фазу и взять реальную и мнимую части

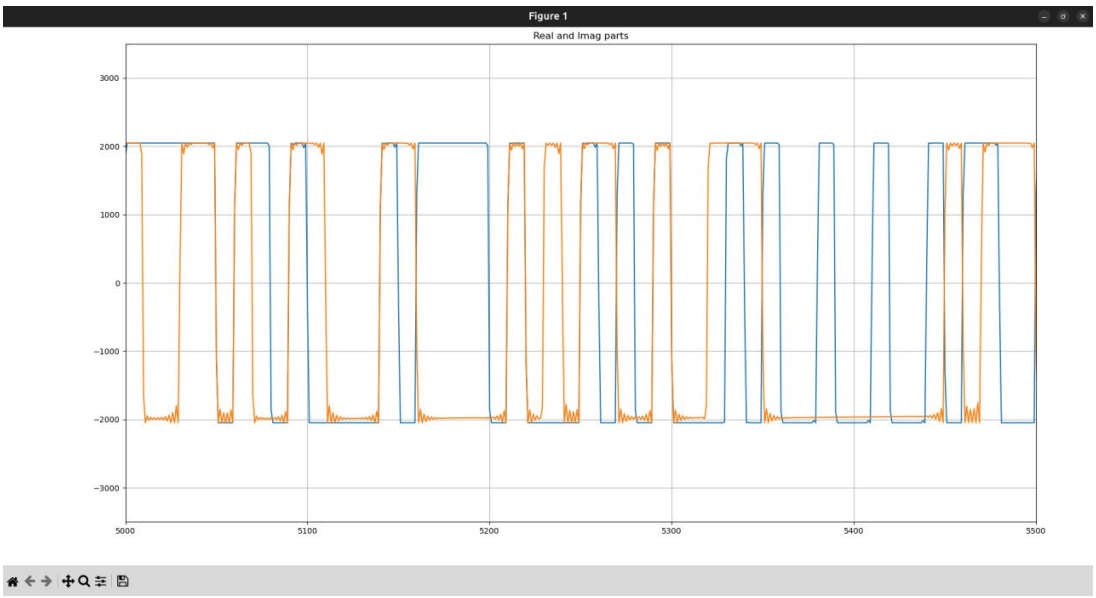


Рис. 21 — Реальная и мнимая части полученных битов

Вывод

Получилось отправить и принять, сгенерированные биты, которые прошли через Mapper, UpSampler, filter, отобразить их амплитуду, фазу, реальную и мнимую части, получить графики.

6 ПРИЕМ QPSK BPSK, ПРИЕМ НА СОГЛАСОВАННЫЙ ФИЛЬТР, ГЛАЗКОВАЯ ДИАГРАММА, ПОИСК ОПТИМАЛЬНОГО ОТСЧЕТНОГО ЗНАЧЕНИЯ И НЕОБХОДИМОСТЬ СИМВОЛЬНОЙ СИНХРОНИЗАЦИИ

Ссылка на GitHub

Цель практики:

Сформировать биты, пропустить их через согласованный фильтр. Построить глазковую диаграмму, в ручную найти оптимальное отсчетное значение

Краткие теоретические сведения

Согласованный фильтр (matched filter) в цифровой обработке сигналов — это линейный фильтр, обеспечивающий максимальное отношение сигнал/шум (SNR) на выходе в момент принятия решения при наличии аддитивного белого гауссовского шума (AWGN).

Для известного сигнала $s(t)$ длительностью T, импульсная характеристика согласованного фильтра задаётся как:

$$h(t) = s(T - t),$$

то есть представляет собой зеркально отражённую во времени копию сигнала $s(t)$, сдвинутую на T.

В дискретной форме, для последовательности отсчётов $s[n]$, $n = 0, 1, \ldots, N - 1$, импульсная характеристика согласованного фильтра:

$$h[n] = s^*[N - 1 - n],$$

где $(\cdot)^*$ обозначает комплексное сопряжение.

Выход согласованного фильтра вычисляется как свёртка входного сигнала $x[n]$ с $h[n]$:

$$y[n] = \sum_{k=0}^{N-1} x[k] \cdot s^*[k - n],$$

что эквивалентно корреляции входного сигнала с ожидаемой формой сигнала.

Согласованный фильтр широко применяется в программно-определяемых радиосистемах (SDR) для обнаружения и демодуляции сигналов с известной структурой (например, пилотных символов, преамбулы или кодов расширения в CDMA).

Глазковая диаграмма (eye diagram) — это визуальное представление цифрового сигнала, полученное путём наложения друг на друга нескольких последовательных тактовых интервалов в осциллографе или программной среде.

Формируется глазковая диаграмма следующим образом: осциллограф или программа захватывает отрезки сигнала длительностью, кратной периоду тактовой частоты T, и отображает их синхронно по фронту тактового сигнала. В результате наложения траекторий сигнала возникает характерная форма, напоминающая «глаз».

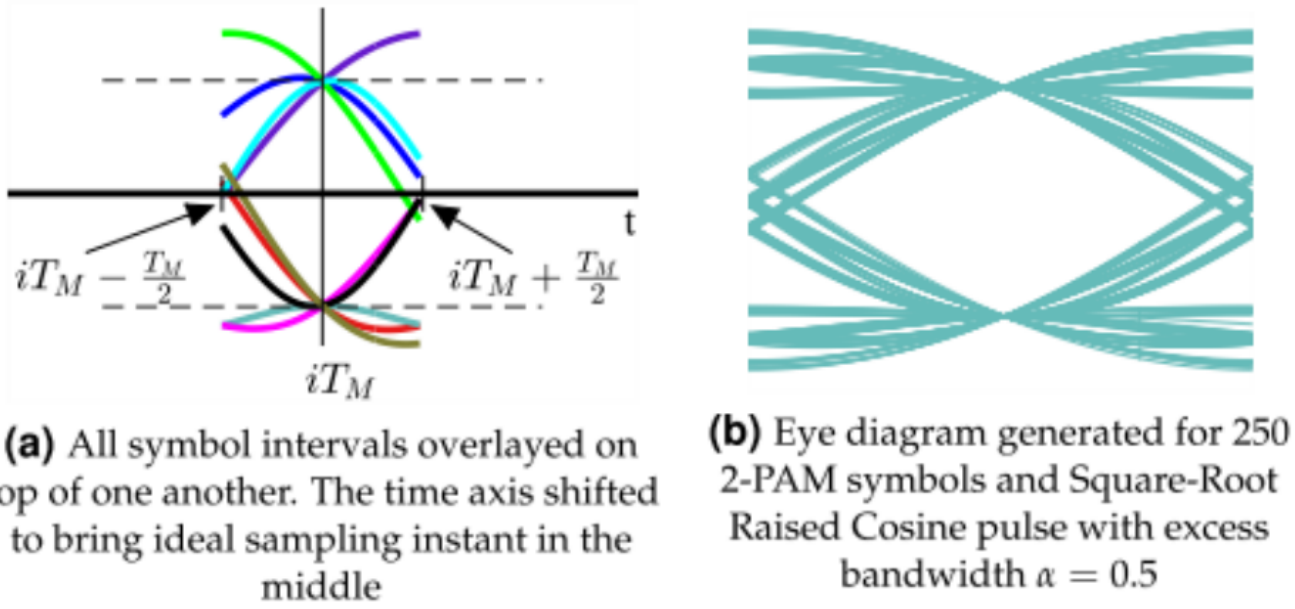
Качество цифровой передачи оценивается по степени раскрытия «глаза»: - чем шире и выше глаз, тем меньше межсимвольная интерференция (ISI) и шум; - закрытый глаз указывает на сильную ISI, джиттер или высокий уровень шума.

Глазковая диаграмма (eye diagramm) дает наглядное представление о форме принимаемого сигнала на выходе согласованного фильтра. Глазковая диаграмма строится в виде накапливающего изображения сигналов на выходе СФ в интервале времени

$$0.5T_s < t < 0.5T_S$$

по большому числу символов

Если изобразить много таких отрезков сигнала на выходе СФ в виде накапливающейся суммы изображений, то получим глазковую диаграмму.



(a) All symbol intervals overlayed on top of one another. The time axis shifted to bring ideal sampling instant in the middle

(b) Eye diagram generated for 250 2-PAM symbols and Square-Root Raised Cosine pulse with excess bandwidth $\alpha = 0.5$

Рис. 22 — Глазковая диаграмма

Выполнение

Согласованный фильтр уже реализован в функции filter

Листинг 6.1 — filter

```
void filter(complex<double> *symbols_ups, int len_symbols_ups, complex<double> *
    impulse, int L) {
    vector<complex<double>> sum(len_symbols_ups, 0);
    for (size_t i = 0; i < (size_t)len_symbols_ups; i++) {
```

```
        for (size_t j = 0; j < (size_t)L && (int)(i-j) >= 0; j++) {
            sum[i] += impulse[j] * symbols_ups[i-j];
        }
    }
    memcpy(symbols_ups, sum.data(), len_symbols_ups * sizeof(complex<double>));
}
```

Чтобы получить глазковую диаграмму, нужно написать наш код на python

Листинг 6.2 — python eye diagram

```
import numpy as np
import matplotlib.pyplot as plt

data = np.fromfile("/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
    int16)

samples = []
for x in range(0, len(data) - 1, 2):
    samples.append(data[x] + 1j * data[x+1])

real = np.real(samples)

sps = 10
segments = []
for i in range(0, len(real) - 2*sps, sps):
    segments.append(real[i:i + 2*sps])

plt.plot(np.array(segments).T)
plt.xlim(7.5, 12.5)
plt.grid(True)
plt.show()
```

Получим такой результат

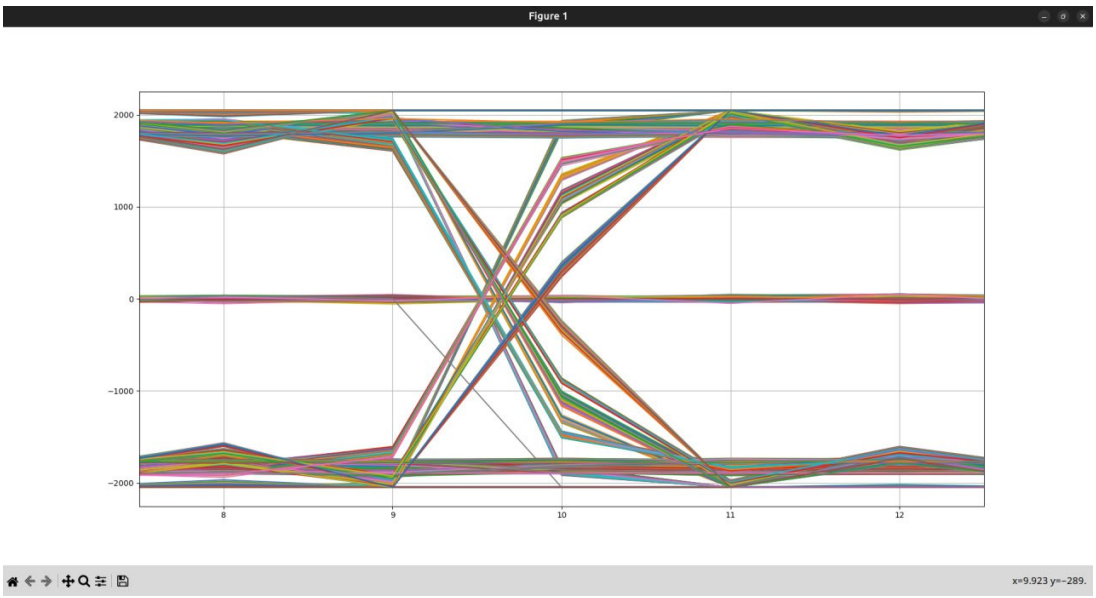


Рис. 23 — Глазковая диаграмма

Поиск оптимального значения отсчетного значения мы будем выполнять вручную. Сперва напишем код для отображения сигнального созвездия

Листинг 6.3 — python eye diagram

```
from matplotlib import pyplot as plt
import numpy as np

rx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
    int16)

samples = []

for x in range(0, len(rx), 2):
    samples.append((rx[x] + 1j * rx[x+1])/np.max(rx))

impulse = np.ones(10)

I = np.real(samples)
Q = np.imag(samples)

i_convolve = np.convolve(I, impulse)
q_convolve = np.convolve(Q, impulse)
i_con_dec = []
q_con_dec = []

for i in range(-1, len(i_convolve), 10):
    i_con_dec.append(i_convolve[i])
    q_con_dec.append(q_convolve[i])

# Убираемшумы
i_con_dec = [x for x in i_con_dec if not (abs(x) <= 1)]
```

```
q_con_dec = [x for x in q_con_dec if not (abs(x) <= 1)]

plt.scatter(i_con_dec, q_con_dec)
plt.axhline()
plt.axvline()
plt.show()
```

Строчки когда я убираю шумы, иногда меняют длину массивов, что приводит к ошибке, поэтому иногдя я их комментирую, и для таких значений у сигнального созвездия будет шум в центре

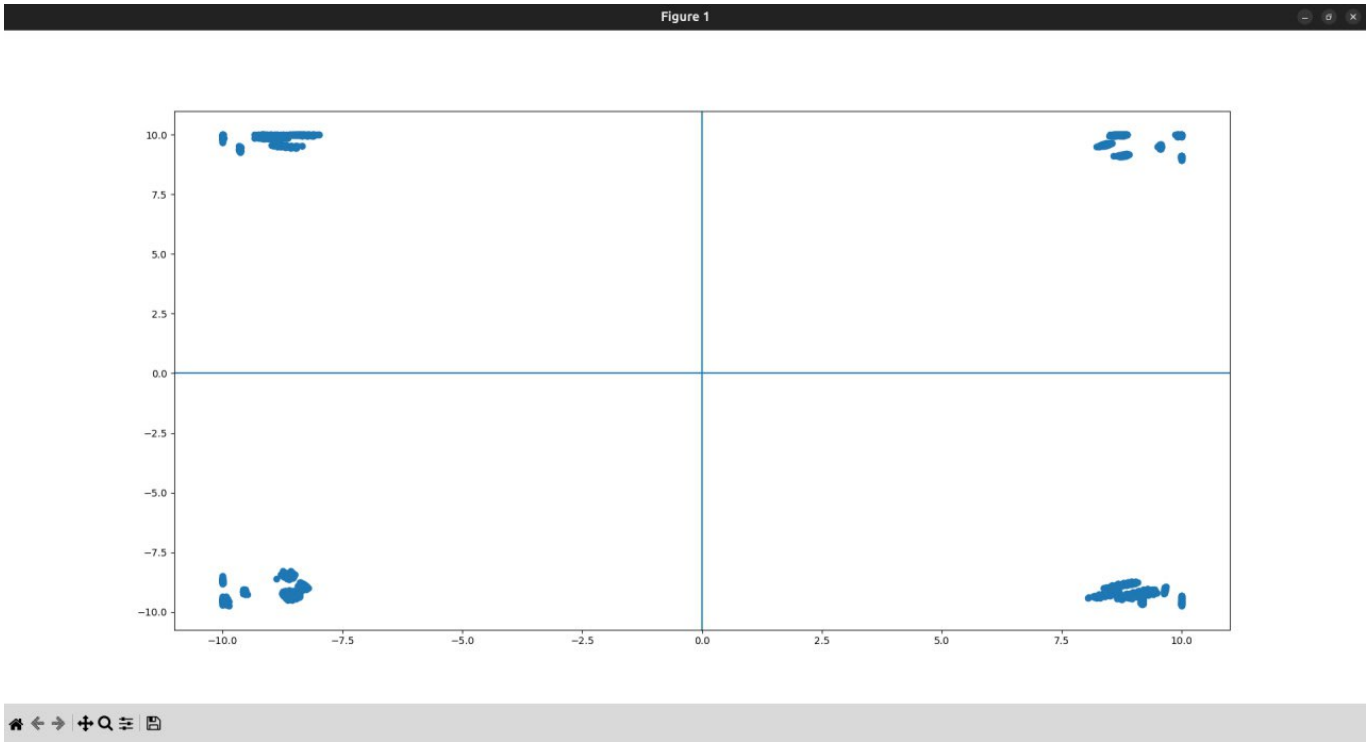


Рис. 24 — Отсчетное значение = -1

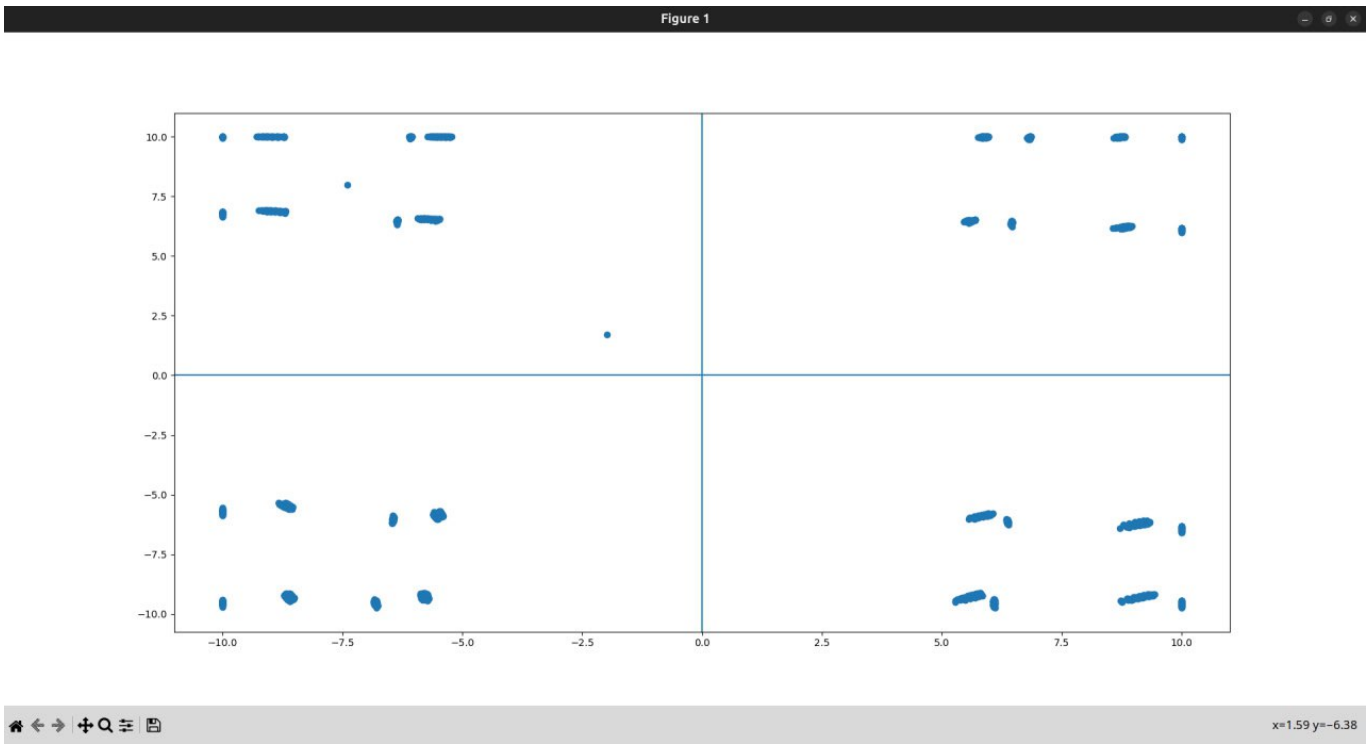


Рис. 25 — Отсчетное значение = 1

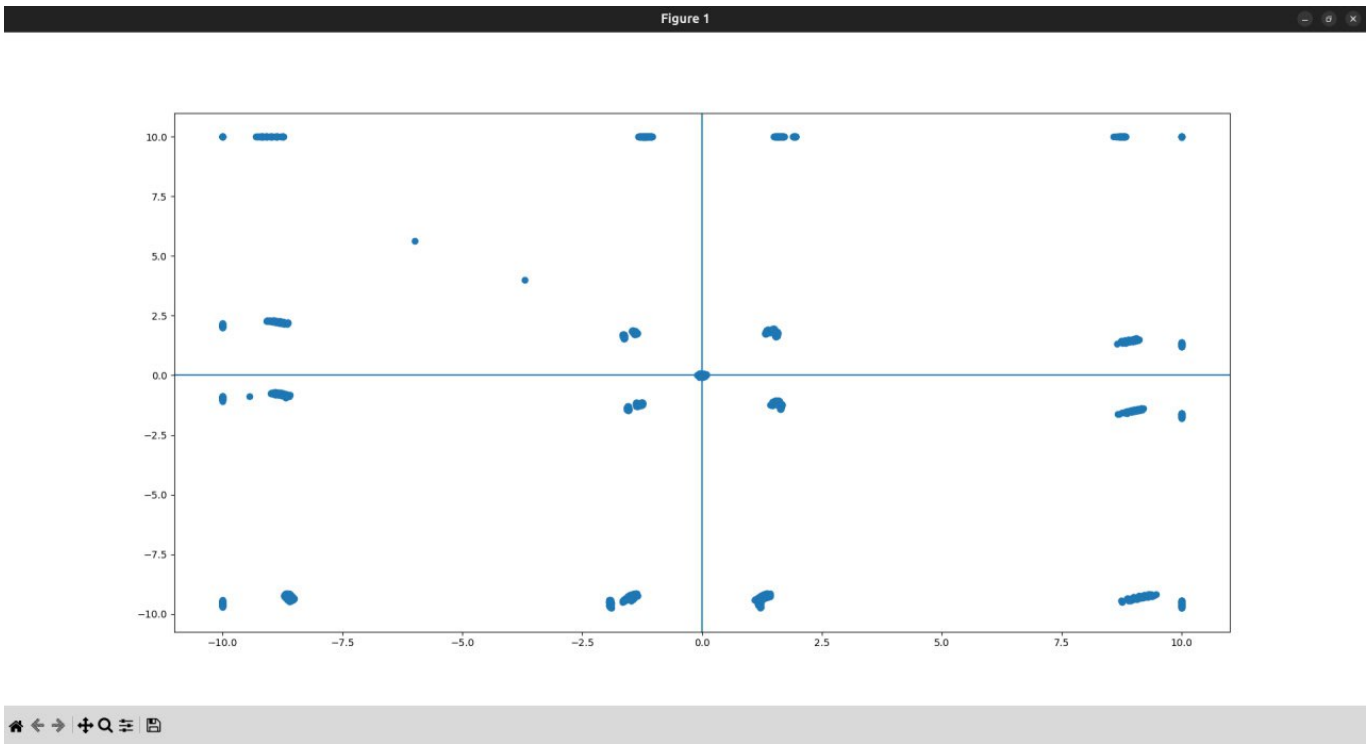


Рис. 26 — Отсчетное значение = 5

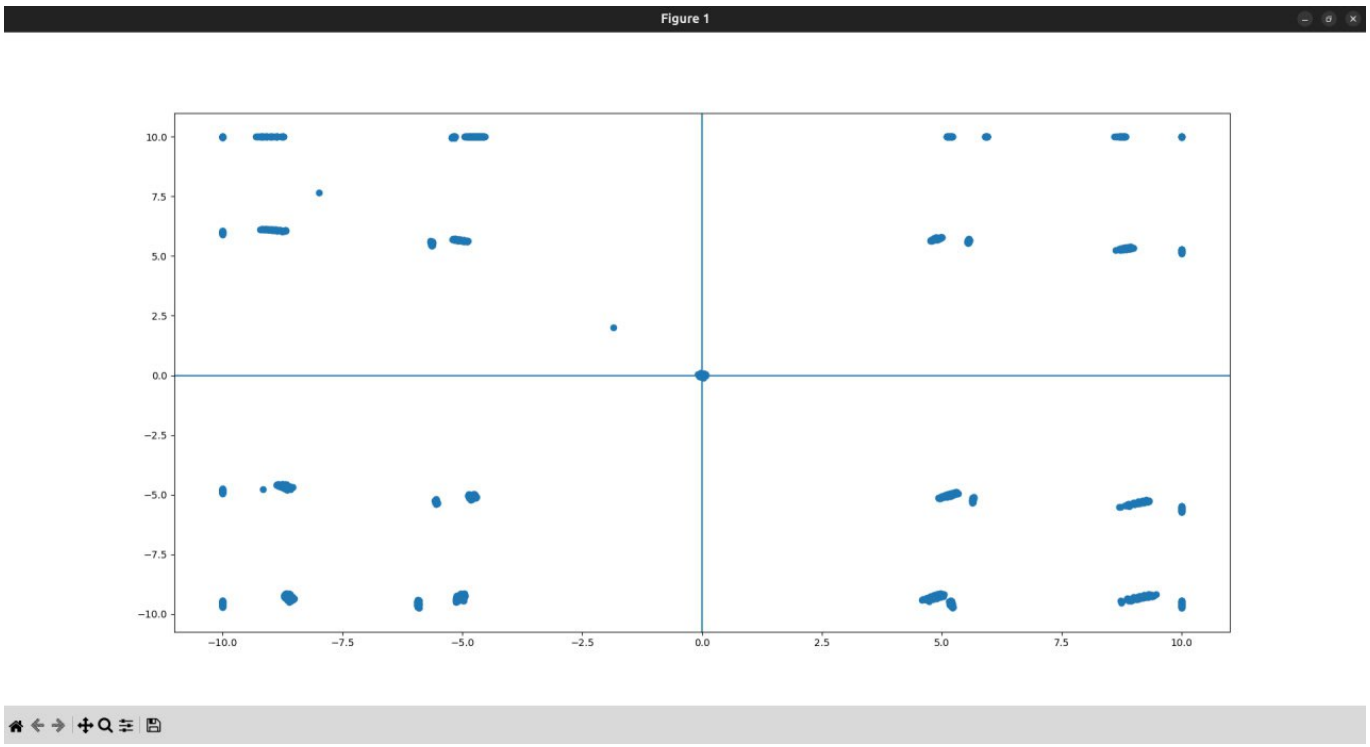


Рис. 27 — Отсчетное значение = 7

По итогу самое лучшее отсчетное значение оказалось -1, сигнальное созвездие с этим значением выглядит лучше всего, более похожее на сигнальное созвездие QPSK

Вывод

Получили глазковую диаграмму, в ручную подобрали отсчетное значения для лучшей схожести с сигнальным созвездием QPSK, получили графики для разных отсчетных значений.

7 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ ДЕТЕКТОРА ВРЕМЕННОЙ ОШИБКИ(СИНХРОНИЗАЦИЯ ПРИЕМНИКА И ПЕРЕДАТЧИКА) НА SDR. НАПИСАНИЕ ФУНКЦИЙ ПЕТЛИ (КОНТУРА) СИНХРОНИЗАЦИИ

Ссылка на GitHub

Цель практики:

Написать функцию которая будет автоматически синхронизировать примник и передатчик.

Краткие теоретические сведения

Оптимальный приемник выполняет прием дискретных сигналов с минимальной вероятностью ошибки. Строится оптимальный приемник по схеме на основе корелляторов или согласованных фильтров.

Оптимальный приемник выбирает наиболее близкое принятому сигналу r(t) из набора возможных колебаний si(t). Это может выполняться вычислением расстояния

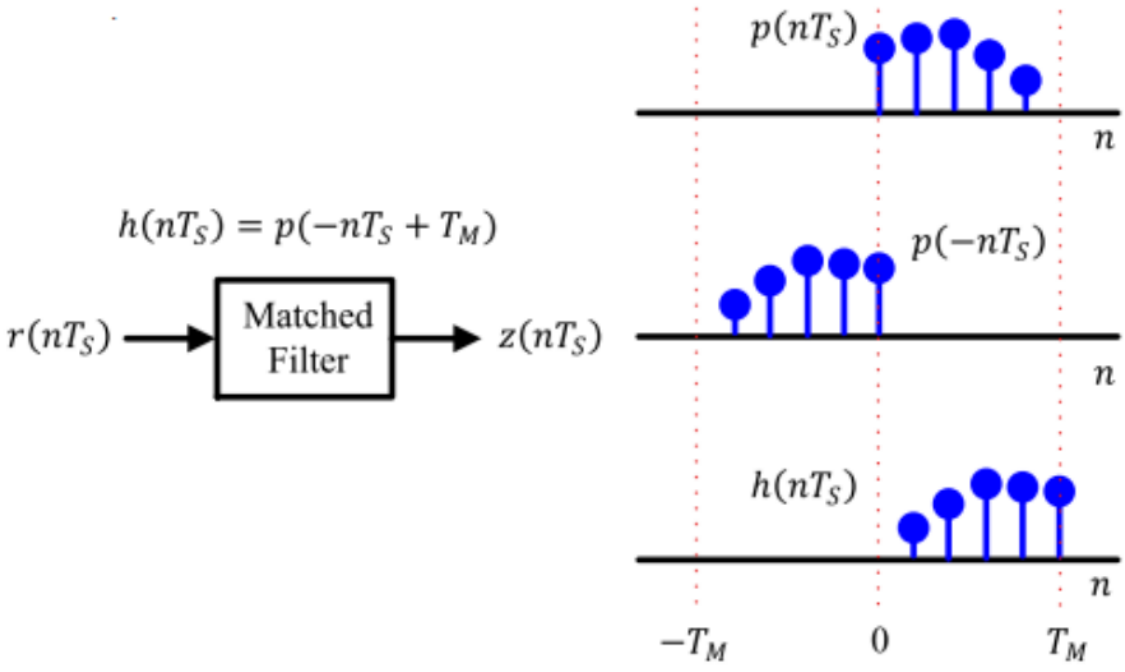


Рис. 28 — Matched filter

Если использовать формирующий фильтр с прямоугольной формой ИХ, то форма ИХ согласованного фильтра также является прямоугольной. Сигнал на выходе СФ вычисляется как свертка отсчетов входного сигнала и ИХ согласованного фильтра.

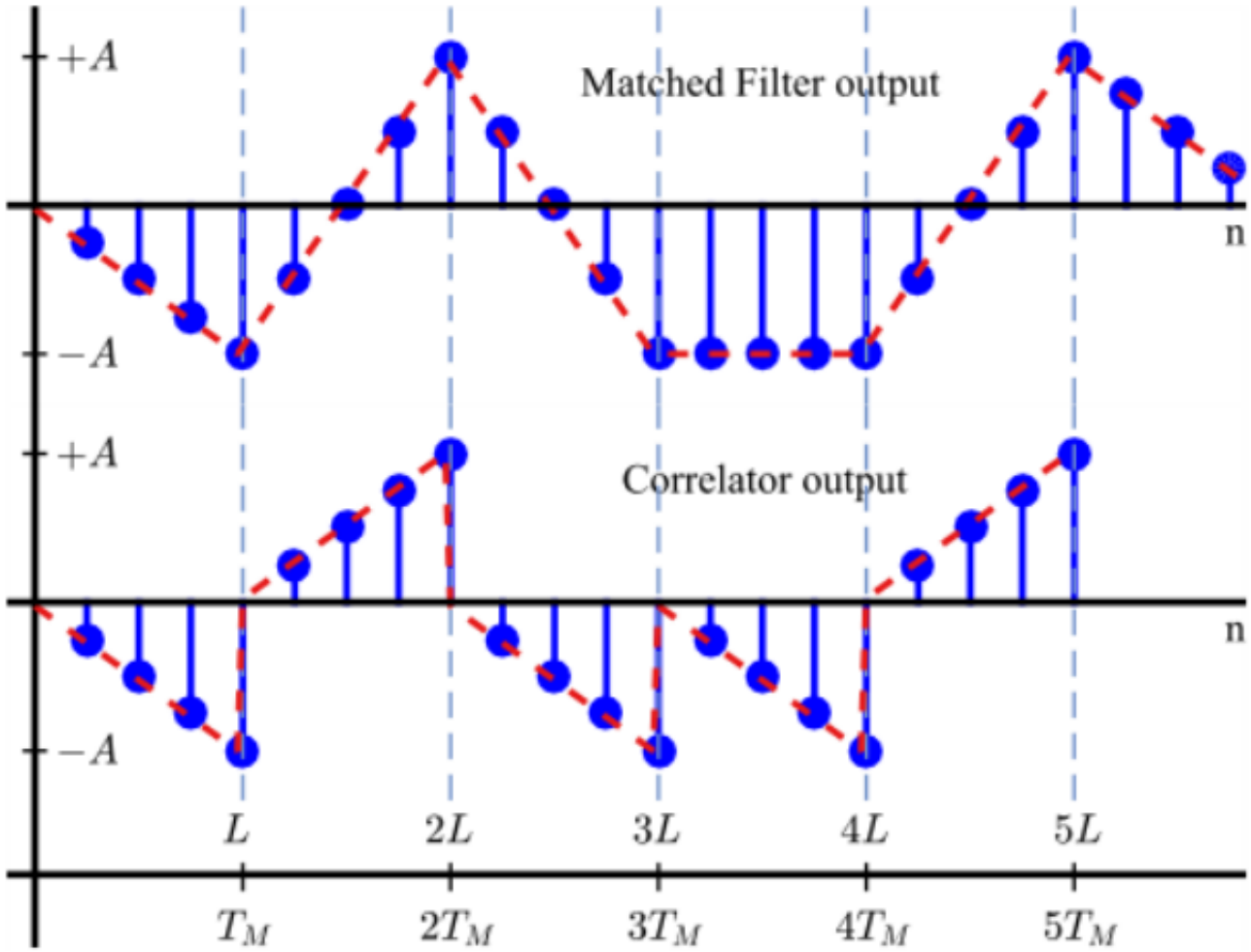


Рис. 29 — Matched filtet/correlator output

Демодулятор должен сделать отсчет сигнала на выходе СФ точно в момент максимума сигнала на выходе СФ. Если время взятия отсчета не совпадает с максимумом, то невозможно принять надежное решение о передаваемом символе.

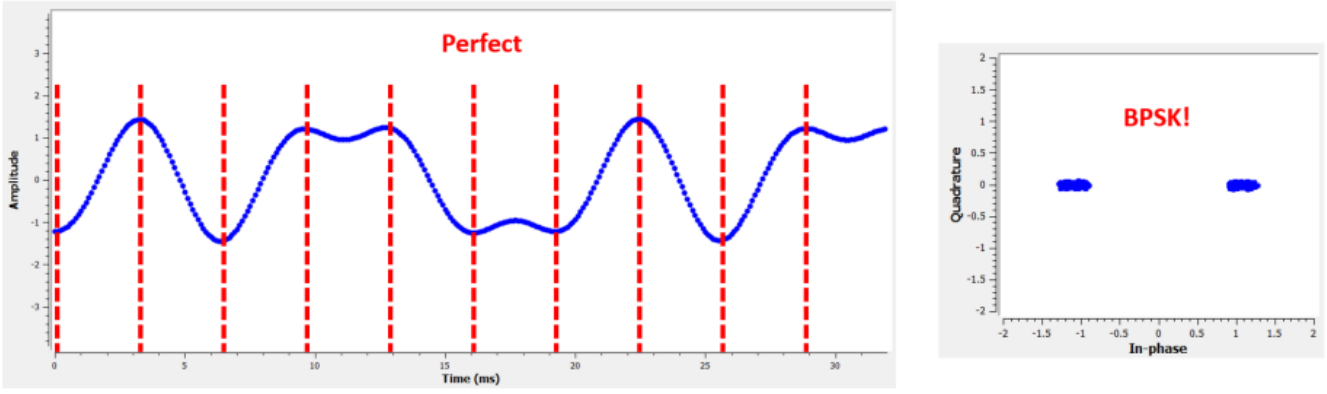


Рис. 30 — Идеальный отсчет

Время взятия отсчета (symbol timing) совпадает с максимумом сигнала на выходе СФ. При смещении времени взятия отсчета берутся значения с выхода СФ не в оптимальные моменты времени, что приводит к нарушению возможности точного определения амплитуды символа.

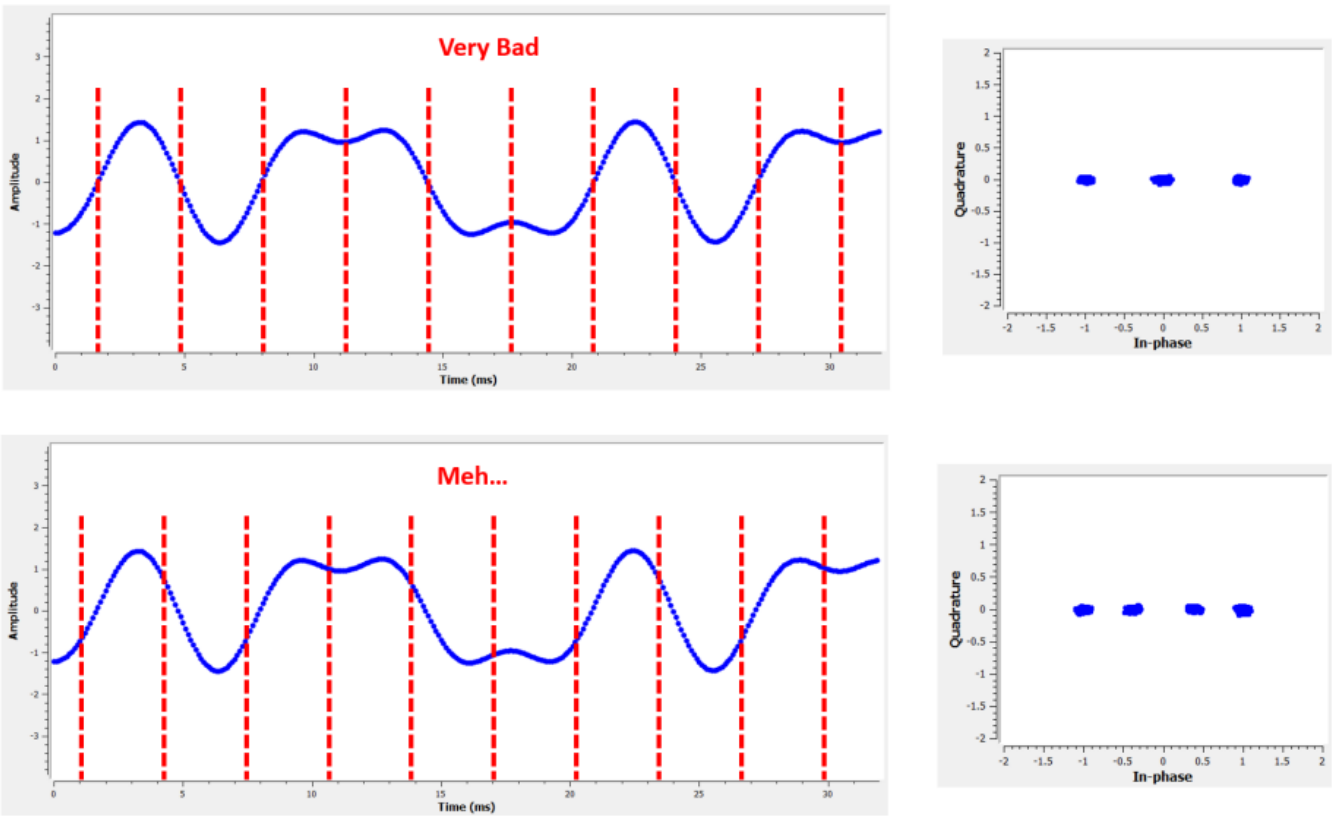


Рис. 31 — Плохой отсчет

Детектор временной ошибки - TED

Для поиска нужного отсчета сначала вычисляется ошибка положения символа при помощи детектора временной ошибки. Детектор использует набор отсчетов и вычисляет ошибку ТЕД e(n), величина которой зависит от смещения максимума выхода СФ. В таблице приведены выражения для вычисления величины ошибки по отсчетам сигнала с выходы СФ.

TED Method	Expression
Zero-crossing (decision-directed)	$e(k) = x((k-1/2)T_s + \hat{\tau})[\hat{a}_0(k-1) - \hat{a}_0(k)] + y((k-1/2)T_s + \hat{\tau})[\hat{a}_1(k-1) - \hat{a}_1(k)]$
Gardner (non-data-aided)	$e(k) = x((k-1/2)T_s + \hat{\tau})[x((k-1)T_s + \hat{\tau}) - x(kT_s + \hat{\tau})] + y((k-1/2)T_s + \hat{\tau})[y((k-1)T_s + \hat{\tau}) - y(kT_s + \hat{\tau})]$
Early-late (non-data-aided)	$e(k) = x(kT_s + \hat{\tau})[x((k+1/2)T_s + \hat{\tau}) - x((k-1/2)T_s + \hat{\tau})] + y(kT_s + \hat{\tau})[y((k+1/2)T_s + \hat{\tau}) - y((k-1/2)T_s + \hat{\tau})]$
Mueller-Muller (decision-directed)	$e(k) = \hat{a}_0(k-1)x(kT_s + \hat{\tau}) - \hat{a}_0(k)x((k-1)T_s + \hat{\tau}) + \hat{a}_1(k-1)y(kT_s + \hat{\tau}) - \hat{a}_1(k)y((k-1)T_s + \hat{\tau})$

Рис. 32 — TED

Зависимость сигнала на выходе TED от величины смещения символа (временной ошибки) называется S-curve, для детектора Гарднера она имеет форму

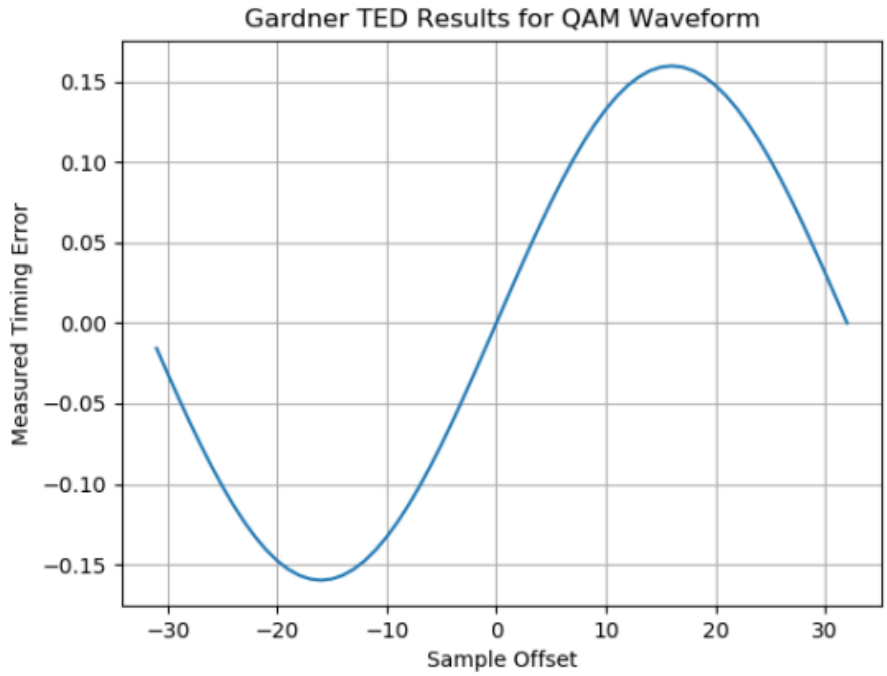


Рис. 33 — Форма TED

Фильтр контура

Фильтр выполнен в виде пропорционально-интегрирующего звена по схеме

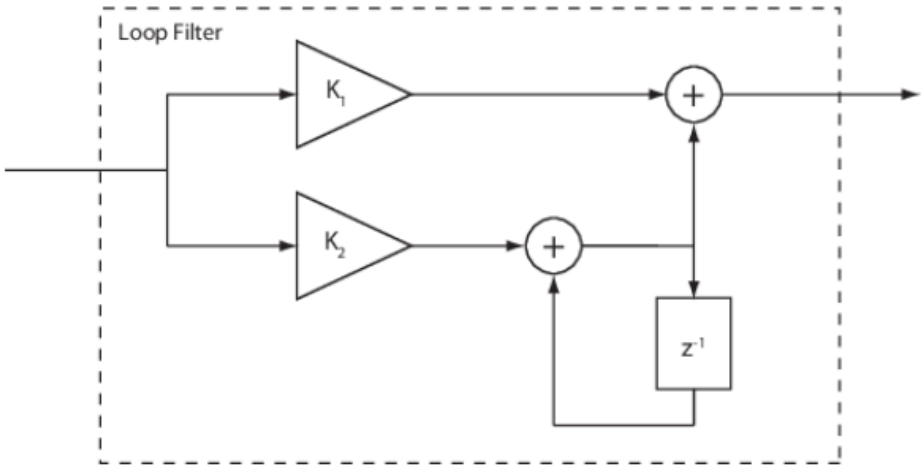


Рис. 34 — Форма TED

Параметры фильтра

$$K_1 = \frac{-4\zeta\theta}{(1 + 2\zeta\theta + \theta^2)K_p}$$

and

$$K_2 = \frac{-4\theta^2}{(1 + 2\zeta\theta + \theta^2)K_p} .$$

The interim term, θ, is given by

$$\theta = \frac{\frac{B_n T_s}{N_{sps}}}{\zeta + \frac{1}{4\zeta}} ,$$

where:

- N_{sps} is the number of samples per symbol.
- ζ is the damping factor.
- $B_n T_s$ is the loop bandwidth (B_n) normalized to the symbol rate (T_s).
- K_p is the detector gain.

Рис. 35 — Параметры фильтра

Программно фильтр реализуется в общем цикле по отсчетам сигнала на выходе СФ.

Начальные значения переменных в ветвях K1 и K2 - p1 и p2 равны 0. Сами величины K1 и K2 вычисляются перед циклом. Нормированная полоса фильтра контура BnTs = 0.01. Также перед циклом задается начальное смещение offset =0 (номер отсчета символа, один из 10).

В цикле по отсчетам сигнала на выходе СФ вычисляется текущее значение ошибки e для текущего смещения n = offset;

e = (I(n+Nsp+Ns) - I(n+Ns)) * I(n+Nsp/2+Ns) + (Q(n+Nsp+Ns) - Q(n+Ns)) * Q(n+Nsp/2+Ns).

Если e - текущая величина ошибки на выходе детектора ошибки (на данном индексе отсчета сигнала на выходе СФ), то в ветви K1 переменная p1 = e*K1. В ветви K2 переменная p2 = p2 + p1 + e*K2. Эта переменая p2 имеет смысл задержки, дробного смещения (offset) на длительности символа. Тут же , если p2>1, ее сбрасывают

while(p2 > 1) p2 = p2 - 1; end.

После этого определяется уточненное смещение в отсчетах (offset), для этого нужно умножить p2 на Nsps и округлить произведение до ближайшего целого, offset = round(p2*Nsps), это будет смещение на данной итерации (оптимальный отсчет текущего символа, индекс в массиве отсчетов сигнала на выходе СФ). На следующем символе снова вычисляется величина ошибки и после фильтрации определяется смещение, по которому находится следующий индекс отсчета offset+Nsps . Таким образом контур сводит ошибку к мимнимуму и определяются оптимальные индексы в массиве отсчетов символов сигнала на выходе СФ.

Выполнение

Напишем функцию Гарднера, которая будет автоматически искать offset

Листинг 7.1 — Функция Гарднера

```
import numpy as np
def gardner(complex_symbols_saw):
    p1 = 0
    p2 = 0
    offset = 0
    BnTs = 0.01
    Nsp = 10 # Samples per symbol
    zeta = np.sqrt(2)/2
    teta = (BnTs/Nsp)/(zeta + 1/(4*zeta))
    css = complex_symbols_saw
    Kp = 0.002
    K1 = (-4 * zeta * teta)/((1 + 2 * zeta * teta + teta ** 2) * Kp)
    K2 = (-4 * teta ** 2)/((1 + 2 * zeta * teta + teta ** 2) * Kp)
    list_of_offset = []

    for n in range(0, len(css)-22):
        err = (np.real(css[n + Nsp + Nsp]) - np.real(css[n + Nsp])) * np.real(css[n +
            Nsp + Nsp//2]) + (np.imag(css[n + Nsp + Nsp]) - np.imag(css[n + Nsp])) *
           np.imag(css[n + Nsp + Nsp//2])
        err = np.real(err)
        p1 = err * K1
        p2 = p2 + p1 + err * K2
        while p2 > 1:
            p2 = p2 - 1
        while p2 < -1:
            p2 = p2 + 1
        offset = np.round(p2 * Nsp)
        list_of_offset.append(offset)

    return int(offset), list_of_offset
```

Используем эту функций в нашем коде

Листинг 7.2 — Функция Гарднера

```
from matplotlib import pyplot as plt
import numpy as np
from auto_offset import gardner

rx = np.fromfile(f"/home/someotb/Files/Code/sdr/pluto/dev/build/rx.pcm", dtype=np.
    int16)

samples = []
```

```
8
9  for x in range(0, len(rx), 2):
10     samples.append((rx[x] + 1j * rx[x+1])/np.max(rx))
11
12 plt.figure()
13 plt.scatter(np.real(samples), np.imag(samples))
14 plt.axhline()
15 plt.axvline()
16
17 impulse = np.ones(10)
18
19 I = np.real(samples)
20 Q = np.imag(samples)
21
22 i_convolve = np.convolve(I, impulse)
23 q_convolve = np.convolve(Q, impulse)
24
25 i_con_dec = []
26 q_con_dec = []
27 list_offset = []
28
29 offset, list_offset = gardner(i_convolve + 1j * q_convolve)
30 print(offset)
31
32 plt.figure()
33 plt.title("offset")
34 plt.plot(np.arange(len(list_offset)), list_offset)
35 plt.ylim([-20, 20])
36 plt.xlim([4060, 4560])
37
38 for i in range(offset, len(i_convolve), 10):
39     i_con_dec.append(i_convolve[i])
40     q_con_dec.append(q_convolve[i])
41
42 # Убираемшумы
43 # i_con_dec = [x for x in i_con_dec if not (abs(x) <= 1)]
44 # q_con_dec = [x for x in q_con_dec if not (abs(x) <= 1)]
45
46 print("lenght i_con_dec: ", len(i_con_dec))
47 print("lenght q_con_dec: ", len(q_con_dec))
48
49 plt.figure()
50 plt.subplot(2,1,1)
51 plt.scatter(i_con_dec, q_con_dec)
52 plt.axhline()
53 plt.axvline()
54 plt.subplot(2,1,2)
55 plt.xlim([5000, 5500])
56 plt.ylim([-30, 30])
```

```
plt.scatter(np.arange(len(i_convolve)), i_convolve)
plt.scatter(np.arange(len(q_convolve)), q_convolve)

plt.show()
```

Вывод

Написал, программную реализация детектора ошибки, исполнил в коде схему Гарднера.

8 РЕАЛЗИАЦИЯ СИМВОЛЬНОЙ СИНХРОНИЗАЦИИ. ПРИМЕНЕНИЕ К СЭМПЛАМ С ADALMPLUTO. РАЗРАБОТКА ГРАФИЧЕСКОГО ИНТЕРФЕЙСА (GUI) ДЛЯ РАБОТЫ С SDR. ОКНА НАСТРОЕК SDR, ВЫВОД РЕЗУЛЬТАТОВ ОБРАБОТКИ СЭМПЛОВ И Т.Д.

Ссылка на GitHub

Цель практики:

Написать алгоритм который будет выполнять символьную синхронизацию приемника и передатчика. Написать графический интерфейс, который будет выводить данные в реальном времени с помощью Dear ImGUI, ImPlot.

Краткие теоретические сведения

Символьная синхронизация это алгоритм, который находит начало полезного сигнала. Я сделал его с помощью PSS. А именно с помощью последовательности Zadoff-Chu. Графический интерфей был написан с помощью библиотеки Dear ImGui, а сопутсвующие графики с помощью дополнения ImPlot.

Символьная синхронизация по Zadoff-Chu

Последовательности Задова-Чу (Zadoff-Chu) — это CAZAC-последовательности (constant amplitude zero autocorrelation): комплексные последовательности с постоянной амплитудой и нулевой периодической автокорреляцией при любом ненулевом сдвиге. В LTE/5G применяются для синхронизации (PSS — Primary Synchronization Signal) и для преамбул случайного доступа (PRACH), потому что приёмник может по пику корреляции точно определить временной сдвиг сигнала, а разные корневые индексы последовательности дают взаимно почти ортогональные (низкая кросс-корреляция) сигналы — это позволяет разным базовым станциям/пользователям использовать разные ZC-последовательности без сильной взаимной интерференции.

Механика: последовательность генерируется как $x_u(n) = e^{-j\pi un(n+1)/N}$ (для нечётного N), где u — корневой индекс, взаимно простой с N. Идеальная нулевая автокорреляция сохраняется только для циклического сдвига внутри одного периода — на практике приёмник использует именно это свойство, ища по корреляции с локальной копией пик, соответствующий временной задержке до передатчика.

Dear ImGui, ImPlot

Dear ImGui — библиотека для GUI в режиме immediate mode: интерфейс не хранится как дерево объектов, а переописывается каждый кадр вызовами функций, которые сразу и рисуют виджет, и возвращают результат взаимодействия. Она не занимается рендерингом и вводом сама — это делает backend (DirectX/OpenGL/Vulkan) и ОС.

ImPlot — надстройка над ImGui той же логики, но для графиков: линии, гистограммы, heatmaps описываются между ‘BeginPlot‘/‘EndPlot‘, данные берутся по указателю без копирования, что удобно для данных, меняющихся каждый кадр.

Выполнение

Напишем функцию, которая будет строить последовательность Zadoff-Chu:

Листинг 8.1 — Функция построения последовательности Zadoff-Chu

```
void build_pss_zadoff_chu(FFT_Context &context, sharedData &sd)
{
    int N_zc = sd.subcarrier - 1;
    int cf = N_zc % 2;
    int q = 1;

    for (int i = 0; i < context.N; ++i)
    {
        context.in[i][0] = 0.0f;
        context.in[i][1] = 0.0f;
    }

    for (int n = 0; n < N_zc; ++n)
    {
        float phase = -(M_PIf * (float)sd.zadoff_chu_u * (float)n * (float)(n + cf +
           2 * q)) / (float)N_zc;
        float sin, cos;
        sin = sinf(phase);
        cos = cosf(phase);
        context.in[n][0] = cos;
        context.in[n][1] = sin;
    }

    context.ifft();
}
```

Она строит последовательность, записывает сэмплы в специальный массив ищ библиотеки fftw3, которая делает быстро преобразование Фурье. И сразу в этой функции делается обратное преобразование Фурье, за это отвечается строка: ‘context.ifft();‘

Листинг 8.2 — Функци корреляции Zadoff-Chu

```
int zadoff_sync(const float *signal_re, const float *signal_im, size_t signal_len,
   const float *zc_re,
                const float *zc_im, size_t zc_len, float *out_corr)
{
    float max_norm = -1.f;
    int best_idx = 0;
```

```
6
7       for (size_t n = 0; n < signal_len - zc_len; ++n)
8       {
9           float sum_re = 0.0f;
10          float sum_im = 0.0f;
11
12          for (size_t k = 0; k < zc_len; ++k)
13          {
14              sum_re += signal_re[n + k] * zc_re[k] + signal_im[n + k] * zc_im[k];
15              sum_im += signal_im[n + k] * zc_re[k] - signal_re[n + k] * zc_im[k];
16          }
17
18          float cur_norm = sum_re * sum_re + sum_im * sum_im;
19
20          if (out_corr)
21              out_corr[n] = cur_norm;
22
23          if (cur_norm > max_norm)
24          {
25              max_norm = cur_norm;
26              best_idx = (int)n;
27          }
28      }
29      return best_idx;
30  }
```

На вход функция принимает сырые const float указатели, это делается для ускорения выполнения алгоритма, потому что корреляция довольно трудоемкий процесс, а в моей системе важна скорость. Иначе, мой код просто не будет успевать обрабатывать буферы сэмплов. На выходе мы получаем индекс в массиве, который имеет наивысший пик корреляции с известной последовтельностью Zadoff-Chu.

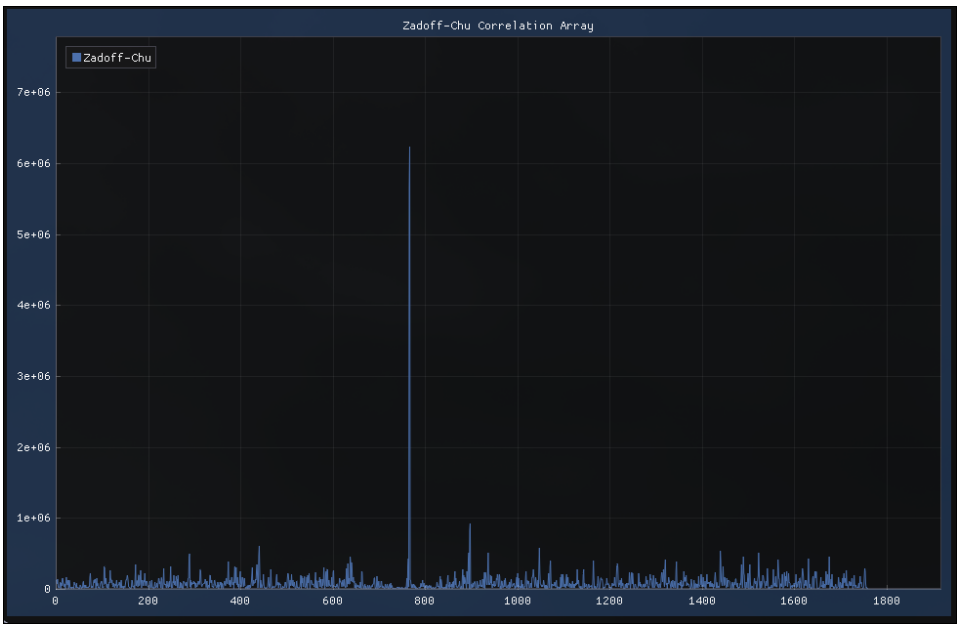


Рис. 36 — График пика корреляции последовательности Zadoff-Chu на всем буфере

Графический интерфейс написан на Dear ImGui, в качестве backend идет OpenGL3, в качестве библиотки для обработки действий пользователя идет GLWF3. Также была выбрана экспериментальная ветка с функцией docking. В этой ветке у внутренних окон есть возможность примагничиваться к крайм экрана и друг к другу. Это позволяет более гибко настраивать рабочую область.

В коде сперва идет инициализация окна и всего прочего, а затем уже создаются окна, подокна и т.д.

Листинг 8.3 — Инициализация окон

```
glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 3);
glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);

#ifdef __APPLE__
    glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
#endif

GLFWwindow* window = glfwCreateWindow(1280, 720, "SDR", nullptr, nullptr);
if (!window) return;
glfwMakeContextCurrent(window);
glfwSwapInterval(1);

IMGUI_CHECKVERSION();
ImGui::CreateContext();
ImPlot::CreateContext();
ImGui::StyleColorsDark();

static std::string ini_path = "../config/imgui.ini";
ImGuiIO &io = ImGui::GetIO();
```

```
io.IniFilename = ini_path.c_str();

ImGui::GetIO().ConfigFlags |= ImGuiConfigFlags_DockingEnable;
ImGui::GetIO().ConfigFlags |= ImGuiConfigFlags_ViewportsEnable;

ImGui_ImplGlfw_InitForOpenGL(window, true);
ImGui_ImplOpenGL3_Init("#version 330");

while (!glfwWindowShouldClose(window))
{
    glfwPollEvents();
    ImGui_ImplOpenGL3_NewFrame();
    ImGui_ImplGlfw_NewFrame();
    ImGui::NewFrame();
    ImGui::DockSpaceOverViewport(0, ImGui::GetMainViewport());

    // Start GUI
```

Вот так выглядит подготовка к созданию окон и контекстов.

Листинг 8.4 — Пример построения графика

```
if (ImGui::Begin("Scatter Raw"))
{
    if (ImPlot::BeginPlot("Raw Samples", ImVec2(ImGui::GetContentRegionAvail())))
    {
        if (!local_raw.empty())
        {
            ImPlotSpec spec;
            spec.Stride = sizeof(std::complex<float>);
            ImPlot::PlotScatter("I/Q", raw_data, raw_data + 1, local_raw.size(), spec
                );
            ImPlot::EndPlot();
        }
        else
        {
            ImPlot::Annotation(0.5, 0.5, ImVec4(1, 0, 0, 1), ImVec2(0, 0), true, "No
                Data");
            ImPlot::EndPlot();
        }
    }
}
ImGui::End();
```

Для создания окон используется ImGui::Begin(); ... ImGui::End();



Рис. 37 — Как выгядит GUI

Вывод

Была написан алогоритм символьной синхронизации, который работает по последовательности Zadoff-Chu. Написан графический интерфейс при помощи библиотки Dear ImGui, на базе OpenGL3 и GLWF3.

9 КОРРЕКЦИЯ ЧАСТОТНОГО СМЕЩЕНИЯ В ОТСЧЁТАХ ПРИНЯТОГО СИГНАЛА ПЕРЕХОД ОТ ОДНОЙ НЕСУЩЕЙ К OFDM , НАЧАЛО ФОРМИРОВАНИЯ И ПРИЕМА OFDM

Ссылка на GitHub

Цель практики:

Перейти от single carrier к OFDM. Сформировать OFDM символы, заполнить буффер этими символами. На приеме раскодировать принятые символы. Осуществить алгоритм коррекции частотного смещения(CFO).

Краткие теоретические сведения

Переход от одной несущей к OFDM

В системах с одной несущей (Single Carrier, SC) каждый передаваемый символ занимает всю доступную полосу частот. При прохождении через многолучевой канал с разбросом задержек τ_{max}, превышающим длительность символа, возникает межсимвольная интерференция (ISI), устранение которой во временной области требует сложного эквалайзера (например, адаптивного КИХ-фильтра большой длины).

В OFDM (Orthogonal Frequency Division Multiplexing) поток данных разбивается на N параллельных потоков, каждый из которых модулирует одну из N ортогональных поднесущих. Формирование временного OFDM-символа из блока комплексных символов данных $X[k]$, $k = 0, \ldots, N-1$, выполняется обратным дискретным преобразованием Фурье (IDFT/IFFT):

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k]\, e^{j2\pi kn/N}, \quad n = 0, \ldots, N-1. \tag{1}$$

Для борьбы с межсимвольной интерференцией к каждому символу спереди добавляется циклический префикс (CP) длиной $N_{CP} > \tau_{max}$ — копия последних N_{CP} отсчётов символа:

$$x_{CP}[n] = x[N+n], \quad n = -N_{CP}, \ldots, -1. \tag{2}$$

CP превращает линейную свёртку сигнала с импульсной характеристикой канала в круговую в пределах длительности символа. Благодаря этому после удаления CP на приёмной стороне и выполнения прямого DFT (FFT) канал в частотной области представляется как поэлементное (плоское) умножение на комплексный коэффициент $H[k]$ для каждой поднесущей:

$$Y[k] = H[k]\, X[k] + W[k], \tag{3}$$

что позволяет выполнять эквализацию однократным делением на оценку $H[k]$, полученную по пилотным поднесущим.

Коррекция частотного смещения (CFO)

Рассогласование частот гетеродинов передатчика и приёмника (Carrier Frequency Offset, CFO) Δf приводит к тому, что принятый дискретный сигнал приобретает линейно нарастающий по времени фазовый набег:

$$r[n] = s[n] \cdot e^{j2\pi\Delta f\, nT_s} + w[n], \tag{4}$$

где $T_s = 1/f_s$ — период дискретизации, f_s — частота дискретизации, $s[n]$ — переданный сигнал без искажений. Некомпенсированное CFO нарушает ортогональность поднесущих и приводит к межканальной интерференции (ICI) после FFT.

Оценка CFO по циклическому префиксу

Поскольку циклический префикс является точной копией хвостовой части OFDM-символа, сдвинутой на N отсчётов, эти два фрагмента сигнала (в отсутствие шума и при статичном CFO) отличаются только фазовым набегом, накопленным за N отсчётов согласно (4). Это позволяет оценить Δf без отдельной обучающей последовательности — корреляцией CP с соответствующим ему фрагментом в конце символа:

$$C = \sum_{k=0}^{N_{CP}-1} r^*[n_0 + k]\, r[n_0 + k + N], \tag{5}$$

где n_0 — начало символа. Фаза этой корреляции связана с CFO соотношением

$$\arg(C) = 2\pi\Delta f\, NT_s, \tag{6}$$

откуда оценка частотного смещения:

$$\hat{\Delta f} = \frac{\arg(C)}{2\pi NT_s} = \frac{\arg(C)\, f_s}{2\pi N}. \tag{7}$$

Для повышения точности оценки корреляционные суммы когерентно накапливаются по всем M символам в обрабатываемом блоке:

$$C_\Sigma = \sum_{m=0}^{M-1} \sum_{k=0}^{N_{CP}-1} r^*[n_m + k]\, r[n_m + k + N], \tag{8}$$

что снижает дисперсию оценки, но требует постоянства Δf на протяжении всего блока.

Поскольку $\arg(\cdot)$ определён на интервале $(-\pi, \pi]$, а фазовый набег в (7) накапливается за N отсчётов, метод даёт однозначную оценку только при

$$|\Delta f| < \frac{f_s}{2N}. \tag{9}$$

При большем смещении возникает неоднозначность (алиасинг оценки).

Коррекция CFO

Компенсация выполняется умножением каждого отсчёта на комплексную экспоненту с противоположным фазовым набегом:

$$\hat{s}[n] = r[n] \cdot e^{-j2\pi\hat{\Delta f}\, nT_s}. \tag{10}$$

Практически это реализуется рекурсивно (по типу цифрового генератора, NCO), чтобы фаза коррекции непрерывно накапливалась от отсчёта к отсчёту и не сбрасывалась на границах обрабатываемых блоков:

$$\varphi[n] = \varphi[n-1] - 2\pi\hat{\Delta} f T_s, \qquad \hat{s}[n] = r[n]\, e^{j\varphi[n]}, \tag{11}$$

с приведением $\varphi[n]$ к интервалу $(-\pi, \pi]$ для численной устойчивости. Такой подход эквивалентен прямому умножению на $e^{-j2\pi\hat{\Delta} f n T_s}$, но не требует хранения абсолютного индекса отсчёта n с начала передачи.

Выполнение

Параметры OFDM который я выбрал:

- **128** поднесущих
- **32** циклический префик
- **27 + 28** защитных нулей по краям OFDM символа(в частотной области)
- **4** пилотные поднесущие(можно менять кол-во)

Функция которая отвечает за построение OFDM символа выглядит так:

Листинг 9.1 — Функция, которая строит один OFDM символ

```
void build_ofdm_symbol(std::vector<uint8_t> &in, FFT_Context &context, const
    sharedData &sd, int &offset)
{
    context.N = sd.subcarrier;
    for (int k = 0; k < context.N; ++k)
    {
        if (sd.is_zeros[k])
        {
            context.in[k][0] = 0;
            context.in[k][1] = 0;
        }
        else if (sd.is_pilot[k])
        {
            context.in[k][0] = 1.0f / std::sqrt(2.0f);
            context.in[k][1] = 0.0f / std::sqrt(2.0f);
```

```
        }
        else
        {
            auto s = map_symbol(in, sd.modul_type_TX, offset);
            context.in[k][0] = s.real();
            context.in[k][1] = s.imag();
        }
    }

    context.ifft();
}
```

В функции используется вектора is_pilot/is_zeros, которые выдают true значение, если в данном индексе должен стоять защитный ноль или пилотная поднесущая. Пилот делается как BPSK символ по стандарту 3GPP. Функция map_symbol возвращает символ, который был сделан из входного массива бит по коду Грея, также по стандарту 3GPP.

Листинг 9.2 — Функция которая сторит символ из битов

```
std::complex<float> map_symbol(std::vector<uint8_t> &in, ModulationType mod, int &
    offset)
{
    if ((size_t)offset > in.size() - 8)
        return {0.0f, 0.0f};

    switch (mod)
    {
    case ModulationType::BPSK:
    {
        float v = 1.0f - 2.0f * in[offset]; offset++;
        return std::complex<float>(v, v) / std::sqrt(2.0f);
    }
    case ModulationType::QPSK:
    {
        float real = 1.0f - 2.0f * in[offset]; offset++;
        float imag = 1.0f - 2.0f * in[offset]; offset++;
        return std::complex<float>(real, imag) / std::sqrt(2.0f);
    }
    case ModulationType::QAM16:
    {
        int b0 = in[offset]; offset++;
        int b1 = in[offset]; offset++;
        int b2 = in[offset]; offset++;
        int b3 = in[offset]; offset++;
        float real = (1.0f - 2.0f * b0) * (2.0f - (1.0f - 2.0f * b2));
        float imag = (1.0f - 2.0f * b1) * (2.0f - (1.0f - 2.0f * b3));
        return std::complex<float>(real, imag) / std::sqrt(10.0f);
    }
    case ModulationType::QAM64:
    {
```

```
31          int b0 = in[offset]; offset++;
32          int b1 = in[offset]; offset++;
33          int b2 = in[offset]; offset++;
34          int b3 = in[offset]; offset++;
35          int b4 = in[offset]; offset++;
36          int b5 = in[offset]; offset++;
37          float real = (1.0f - 2.0f * b0) * (4.0f - (1.0f - 2.0f * b2) * (2.0f - (1.0f
                - 2.0f * b4)));
38          float imag = (1.0f - 2.0f * b1) * (4.0f - (1.0f - 2.0f * b3) * (2.0f - (1.0f
                - 2.0f * b5)));
39          return std::complex<float>(real, imag) / std::sqrt(42.0f);
40      }
41      case ModulationType::QAM256:
42      {
43          int b0 = in[offset]; offset++;
44          int b1 = in[offset]; offset++;
45          int b2 = in[offset]; offset++;
46          int b3 = in[offset]; offset++;
47          int b4 = in[offset]; offset++;
48          int b5 = in[offset]; offset++;
49          int b6 = in[offset]; offset++;
50          int b7 = in[offset]; offset++;
51
52          float real = (1.0f - 2.0f * b0) * (8.0f - (1.0f - 2.0f * b2) * (4.0f - (1.0f
                - 2.0f * b4) * (2.0f - (1.0f - 2.0f * b6))));
53
54          float imag = (1.0f - 2.0f * b1) * (8.0f - (1.0f - 2.0f * b3) * (4.0f - (1.0f
                - 2.0f * b5) * (2.0f - (1.0f - 2.0f * b7))));
55
56          return std::complex<float>(real, imag) / std::sqrt(170.0f);
57      }
58
59      default:
60          throw std::runtime_error("Unsupported mod");
61      }
62  }
```

Также у меня есть функция cfo_est, которая отвечает за коррекцию частотного смещения в пределах 7.5 кГц, так как у меня при sample rate = 1.92e6 Гц и кол-во поднесущих равному 128, получается что каждая поднесущая расположена друг от друга на 15 кГц, отсюда и предел в 7.5 кГц.

Листинг 9.3 — Функция коррекции частотного смещения

```
1  std::vector<std::complex<float>> cfo_est(const std::vector<std::complex<float>> &
       signal, sharedData &sd)
2  {
3      int N = sd.subcarrier;
4      int CP = sd.cyclic_prefex;
```

```
    int sym_len = N + CP;
    float fs = sd.sample_rate;

    int n_symbols = signal.size() / sym_len;
    if (n_symbols < 1)
        return {};

    std::complex<float> corr_sum = 0;
    for (int sym = 0; sym < n_symbols; sym++)
    {
        int sym_start = sym * sym_len;
        std::complex<float> corr = 0;
        for (int k = 0; k < CP; k++)
            corr += conj(signal[sym_start + k]) * signal[sym_start + k + N];
        corr_sum += corr;
    }

    float epsilon = std::arg(corr_sum) / (2 * M_PI);
    float delta_f = epsilon * fs / N; // НасколькосимволотклоняетсяотэталонапофазевГц

    if ((int)sd.cfo_offset.size() > sd.mtu)
    {
        sd.cfo_offset.erase(sd.cfo_offset.begin());
        sd.cfo_offset.push_back(delta_f);
    }
    else
        sd.cfo_offset.push_back(delta_f);

    std::vector<std::complex<float>> corrected(signal.size());
    for (size_t n = 0; n < signal.size(); n++)
    {
        corrected[n] = signal[n] * std::polar(1.0f, sd.current_phase);
        sd.current_phase -= 2 * M_PI * delta_f / fs;
        if (sd.current_phase < -M_PI) sd.current_phase += 2 * M_PI;
        if (sd.current_phase > M_PI)  sd.current_phase -= 2 * M_PI;
    }

    return corrected;
}
```

Функция оценивает частотное смещение по фазовому сдвигу между циклическим префиксом и его копией в конце символа, а затем разворачивает фазу каждого отсчёта на компенсирующий угол.

Сначала сигнал режется на символы длиной ‘N + CP‘ каждый. Для каждого символа берутся первые ‘CP‘ отсчётов (это циклический префикс) и сравниваются с отсчётами, которые находятся ровно на ‘N‘ позиций дальше — это как раз тот кусок символа, копией которого является CP. Сравнение делается через ‘conj(signal[k]) * signal[k+N]‘ — это комплексное произведение, которое одновременно даёт и то, насколько похожи два отсчёта по амплитуде, и на какой угол один повёрнут относительно другого по фазе. Все такие произведения внутри символа складываются, и все символы

блока тоже складываются в одну сумму ‘corr_sum‘ — это накопление увеличивает точность оценки, потому что случайный шум при суммировании частично гасится, а полезный фазовый сдвиг (если он одинаковый во всех символах) складывается когерентно.

Дальше ‘arg(corr_sum)‘ берёт угол этого накопленного комплексного числа — это и есть средний фазовый сдвиг между CP и его оригиналом, накопленный за ‘N‘ отсчётов. Деление на ‘N‘ и умножение на ‘fs‘ переводит этот угол из ”радиан за N отсчётов”в физическую величину — герцы смещения частоты, ‘delta_f‘.

Буфер ‘sd.cfo_offset‘ — это просто скользящее окно последних оценок ‘delta_f‘, ограниченное размером ‘mtu‘, старые значения выбрасываются с начала. В приведённом коде оно нигде не используется дальше, то есть накопленная история пока просто хранится, а не влияет на компенсацию.

Сама компенсация — это отдельный проход по всем отсчётам сигнала. Каждый отсчёт умножается на ‘exp(j * current_phase)‘ (через ‘polar(1.0, phase)‘, что и есть комплексное число единичной длины с этим углом). После каждого отсчёта фаза ‘current_phase‘ уменьшается на маленький шаг ‘2π * delta_f / fs‘ — это и есть та же логика, что ”рассинхронизация растёт с каждым отсчётом только в обратную сторону, компенсирующую эту рассинхронизацию. Условия с ‘-M_PI‘/‘M_PI‘ просто не дают углу уйти за пределы одного оборота окружности, потому что ‘polar‘ работает с любым углом одинаково, а держать угол в разумных пределах — вопрос численной аккуратности.

Вывод

Был осуществлен переход от SC к OFDM. Написан алгоритм коррекции частотного смещения.

10 ФОРМИРОВАНИЯ И ПРИЕМ OFDM. ПОСТРОЕНИЕ СПЕКТРА РАДИОСИГНАЛА (ПРЕОБРАЗОВАНИЕ ФУРЬЕ). ИЗУЧЕНИЕ РАБОТЫ БИБЛИОТЕКИ FFTW3.

Ссылка на GitHub

Цель практики:

Сформировать, отправить и принять OFDM символы. Построить спектр радиосигнала. Использовать библиотеку fftw3 для FFT.

Краткие теоретические сведения

Формирование буффера

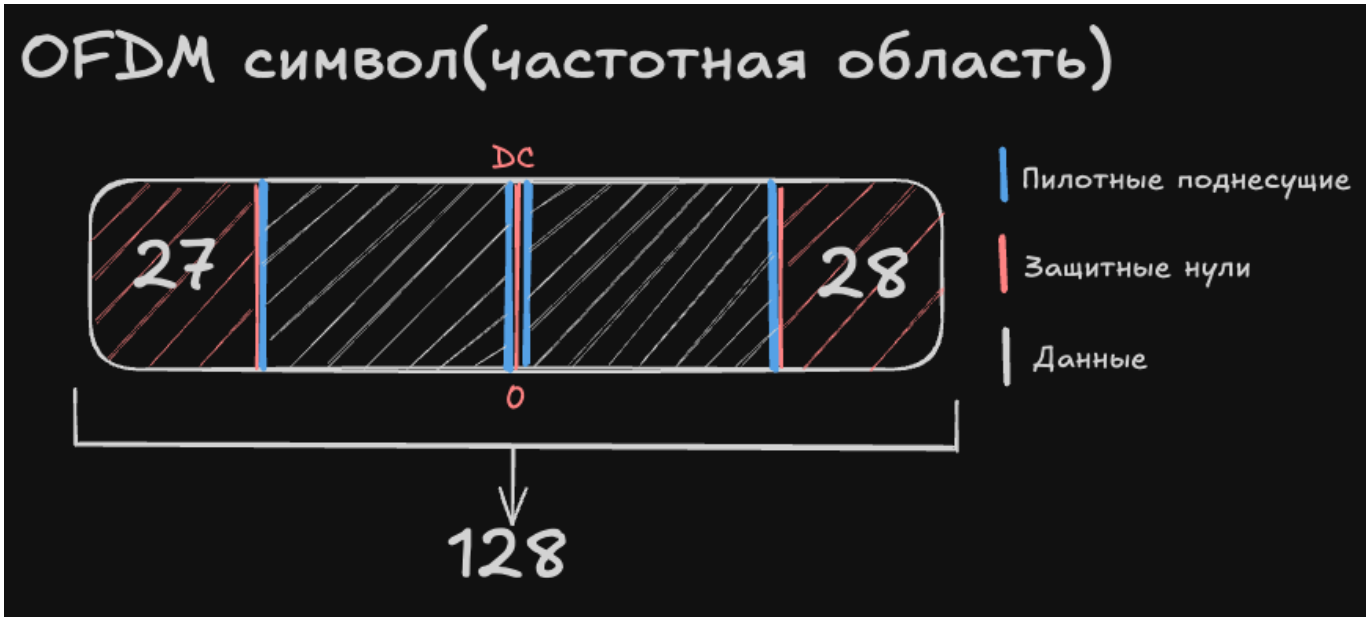


Рис. 38 — Как выгядит OFDM символ в частотной области

Такими OFDM символами мы заполняем буффер размером 1920 символов. Туда влезает 1920 / 128 = 15 OFDM символов, но не забываем что у нас еще есть PSS Zadoff-Chu, который занимает 128 символов, то есть под OFDM символы у нас уходит 1920 - 128 символов. Но, также у нас есть CP - cyclic prefex размером 32 символа, значит размер передоваемого OFDM символа будет 128 + 32 = 160, отсюда общее кол-во OFDM символов котрое мы сможем передать это 1920 / 160 = 12 - 1(под Zadoff-Chu)

Построение спектра. Библиотека fftw3

Чтобы построить спектр радиосигнала нужно провести прямое преобразовние фурье над буфером, чтобы перевести из временной области в частотную. В этом мне поможет библиотека fftw3, которая оптимизирована под работу алгоритма Фурье.

Выполнение

Класс для работы с fftw3

Я уже показывал как формируется OFDM-символ в предыдущем отчете, так что сильно тут останавливаться не будем, перейдем к спектру сигнала. Для удобства я создал отдельный класс для работы с fftw3:

Листинг 10.1 — Класс для работы с fftw3

```
class FFT_Context
{
    private:
        fftwf_plan plan_forward;
        fftwf_plan plan_backward;

    public:
        int N;
        fftwf_complex *in;
        fftwf_complex *out;
        FFT_Context(int n);
        ~FFT_Context();
        void fft();
        void ifft();
};

FFT_Context::FFT_Context(int n) : N(n)
{
    in = fftwf_alloc_complex(N);
    out = fftwf_alloc_complex(N);

    plan_forward = fftwf_plan_dft_1d(N, in, out, FFTW_FORWARD, FFTW_MEASURE);
    plan_backward = fftwf_plan_dft_1d(N, in, out, FFTW_BACKWARD, FFTW_MEASURE);
}

FFT_Context::~FFT_Context()
{
    fftwf_destroy_plan(plan_forward);
    fftwf_destroy_plan(plan_backward);
    fftwf_free(in);
    fftwf_free(out);
}

void FFT_Context::fft()
```

```
35 {
36     fftwf_execute(plan_forward);
37 }
38
39 void FFT_Context::ifft()
40 {
41     fftwf_execute(plan_backward);
42
43     float scale = 1.0f / N;
44     for (int i = 0; i < N; ++i)
45     {
46         out[i][0] *= scale;
47         out[i][1] *= scale;
48     }
49 }
```

Объект класса создается с входным параметром, размер ожидаемого FFT, есть функции для прямого и обратного преобразования, внутренние массивы fftw3 создаются и удаляются автоматически через конструктор и деструктор. Есть встроенный scale после обратного преобразования Фурье.

Спектр радиосигнала

Спектр делается с помощью той же библиотеки fftw3

Листинг 10.2 — Функция для построения спектра

```
1  void spectrum(std::vector<std::complex<float>> &in_signal, std::vector<float> &
       shifted_magnitude, std::vector<float> &argument, FFT_Context &context)
2  {
3      std::vector<float> magnitude(in_signal.size(), 0);
4
5      for (size_t i = 0; i < in_signal.size(); ++i)
6      {
7          context.in[i][0] = std::real(in_signal[i]) / 32768.0;
8          context.in[i][1] = std::imag(in_signal[i]) / 32768.0;
9      }
10
11     context.fft();
12
13     for (size_t i = 0; i < in_signal.size(); ++i)
14     {
15         float real = context.out[i][0];
16         float imag = context.out[i][1];
17         magnitude[i] = 20.0 * log10(sqrt(real * real + imag * imag) / in_signal.size
               ());
18         argument[i] = atan2(imag, real);
19     }
20
21     for (size_t i = 0; i < in_signal.size() / 2; ++i)
```

```
    {
        shifted_magnitude[i] = magnitude[i + in_signal.size() / 2];
        shifted_magnitude[i + in_signal.size() / 2] = magnitude[i];
    }
}
```

Функция выполняет прямое преобразование Фурье, затем берет амплитуду сигнала и переводит ее в dB, также она берет phase. Функция также сдвигает частоты в более привычное расположение, так как после FFT частоты расположены так: [0, +freq, -freq], функция делает их такими: [-freq, 0, +freq]. Вот как это выглядит в GUI, в правом нижнем углу спектр сигнала QPSK:

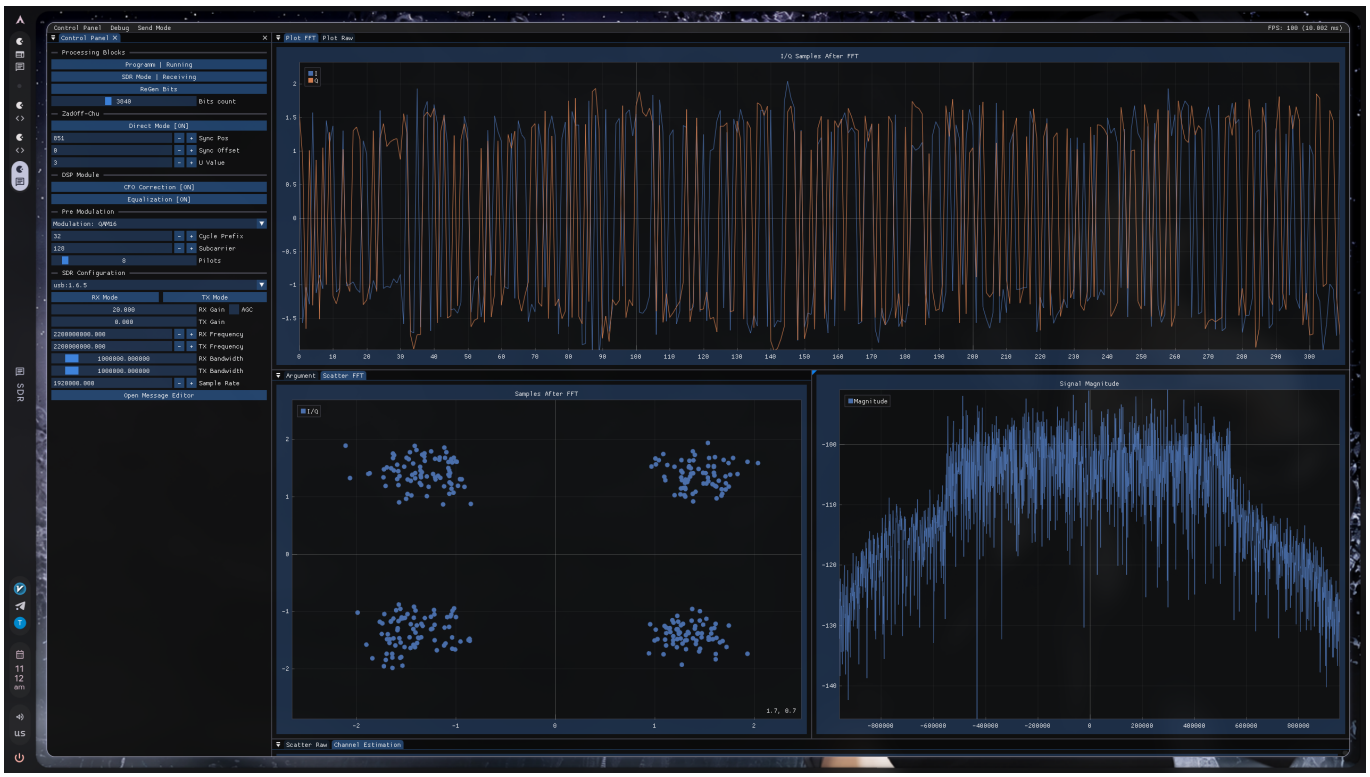


Рис. 39 — Спектр сигнала

Вывод

Был написал класс для удобной работы с fftw3, функция построения спектра, вывод его в GUI.

11 РЕАЛИЗАЦИЯ КОРРЕКЦИИ ЧАСТОТНОГО СМЕЩЕНИЯ ПО ПИЛОТАМ

Ссылка на GitHub

Цель практики:

Реализовать алгоритм частотной коррекции по пилотам, экавализация.

Краткие теоретические сведения

Эквализация по пилотам

После удаления циклического префикса и выполнения прямого DFT над принятым OFDM-символом каждая поднесущая описывается моделью

$$Y[k] = H[k]\,X[k] + W[k], \quad k = 0, \ldots, N-1, \tag{12}$$

где $H[k]$ — комплексный коэффициент передачи канала на k-й поднесущей (амплитудное затухание и фазовый сдвиг, вызванные многолучевым распространением), $X[k]$ — переданный символ, $W[k]$ — аддитивный шум. Поскольку канал частотно-селективен, $H[k]$ различен для разных поднесущих, и для восстановления $X[k]$ его необходимо оценить и скомпенсировать отдельно на каждой поднесущей.

Оценка канала по пилотам

На части поднесущих (пилотных, с индексами $k \in \mathcal{P}$) передаются символы $X_p[k]$, заранее известные приёмнику. Это позволяет получить прямую оценку канала на этих поднесущих:

$$\hat{H}[k] = \frac{Y[k]}{X_p[k]}, \quad k \in \mathcal{P}. \tag{13}$$

Поскольку пилоты занимают не все поднесущие (иначе не осталось бы места под полезные данные), оценка канала для промежуточных поднесущих $k \notin \mathcal{P}$ получается интерполяцией между соседними пилотными оценками (линейной или иного порядка).

Компенсация искажений (эквализация)

Зная оценку $\hat{H}[k]$ для каждой поднесущей, искажение канала устраняется делением принятого символа на эту оценку:

$$\hat{X}[k] = \frac{Y[k]}{\hat{H}[k]}. \tag{14}$$

Эта операция одновременно компенсирует и амплитудное затухание, и фазовый сдвиг, внесённые каналом на данной конкретной поднесущей, что возможно только благодаря диагонализации канала в частотной области, обеспеченной циклическим префиксом.

Отличие от коррекции CFO

Эквализация по пилотам и коррекция частотного смещения решают разные задачи. CFO вносит фазовый сдвиг, одинаково нарастающий во времени для всех поднесущих одновременно и не зависящий от частотной характеристики канала, в то время как $H[k]$ отражает избирательные по частоте искажения, вносимые самим каналом. Если CFO не скомпенсировано до FFT, вносимая им фазовая ошибка исказит и саму оценку $\hat{H}[k]$ по пилотам, поскольку эквалайзер интерпретирует эту ошибку как часть характеристики канала.

Выполнение

Была написана функция эквализации:

Листинг 11.1 — Фунция эквализации

```
void equalization(std::vector<std::complex<float>> &in_signal, sharedData &sd, std::
    vector<std::complex<float>> &out_signal)
{
    out_signal.clear();
    out_signal.reserve(in_signal.size());

    int subcarrar = sd.subcarrier;
    int num_symbols = in_signal.size() / subcarrar;

    for (int i = 0; i < num_symbols; ++i)
    {
        std::vector<std::complex<float>> H(subcarrar, {0.0f, 0.0f});
        std::vector<std::complex<float>> equalized(subcarrar, {0.0f, 0.0f});
        const std::complex<float> known_pilot_conj(1.0f / std::sqrt(2.0f), 0.0f / std
            ::sqrt(2.0f));

        for (int p_idx : sd.pilot_idxs)
        {
            std::complex<float> received_pilot = in_signal[i * subcarrar + p_idx];
            H[p_idx] = received_pilot * known_pilot_conj;
        }

        for (size_t p = 0; p + 1 < sd.pilot_idxs.size(); ++p)
        {
            int k1 = sd.pilot_idxs[p];
            int k2 = sd.pilot_idxs[p + 1];

```

```
26              std::complex<float> H1 = H[k1];
27              std::complex<float> H2 = H[k2];
28
29              float phi1 = std::arg(H1);
30              float phi2 = std::arg(H2);
31
32              float delta_phi = phi2 - phi1;
33
34              if (delta_phi > M_PIf)
35                  delta_phi -= 2 * M_PI;
36
37              if (delta_phi < -M_PIf)
38                  delta_phi += 2 * M_PI;
39
40              for (int k = k1 + 1; k < k2; ++k)
41              {
42                  if (sd.is_zeros[k])
43                      continue;
44
45                  float alpha = float(k - k1) / float(k2 - k1);
46                  H[k] = std::polar(std::abs(H1) + alpha * (std::abs(H2) - std::abs(H1)
                        ), phi1 + alpha * delta_phi);
47              }
48          }
49
50          for (int k = 0; k < sd.pilot_idxs.front(); ++k)
51              H[k] = H[sd.pilot_idxs.front()];
52
53          for (int k = sd.pilot_idxs.back() + 1; k < subcarrar; ++k)
54              H[k] = H[sd.pilot_idxs.back()];
55
56          {
57              std::lock_guard<std::mutex> lock(sd.sync.channel_estimation_mutex);
58              if (sd.channel_estimation.size() >= (size_t)sd.mtu)
59                  sd.channel_estimation.erase(sd.channel_estimation.begin(),
60                                              sd.channel_estimation.begin() + sd.
                                                    subcarrier);
61              for (const auto& h : H)
62                  sd.channel_estimation.push_back(h);
63          }
64
65          for (int k = 0; k < subcarrar; ++k)
66          {
67              if (sd.is_zeros[k] || std::abs(H[k]) < 1e-6f)
68                  equalized[k] = {0, 0};
69              else
70                  equalized[k] = in_signal[i * subcarrar + k] / H[k];
71          }
72
```

```
        for (size_t k = 0; k < equalized.size(); ++k)
        {
            if (sd.is_zeros[k] || sd.is_pilot[k])
                continue;

            out_signal.push_back(equalized[k]);
        }
    }
}
```

Функция эквализации

Функция `equalization` выполняет коррекцию частотно-избирательных искажений канала для каждого OFDM-символа входного буфера отдельно.

Для каждого символа сначала на пилотных поднесущих вычисляется прямая оценка канала $H[k] = Y[k] \cdot \overline{X_p}[k]$, где $Y[k]$ — принятое значение пилота, $\overline{X_p}[k]$ — сопряжённое известное значение пилота. Далее оценка канала для поднесущих между соседними пилотами получается линейной интерполяцией отдельно по модулю и по фазе $H[k]$, с корректной обработкой перехода фазы через $\pm\pi$. Поднесущие до первого и после последнего пилота получают оценку, равную ближайшему крайнему пилоту (экстраполяция константой).

Полученная оценка канала сохраняется в общий буфер `sd.channel_estimation` (с ограничением по размеру `mtu`) для последующего использования, например при построении графиков АЧХ/ФЧХ канала.

Компенсация искажений выполняется делением принятого символа на оценку канала: $\hat{X}[k] = Y[k]/H[k]$. Нулевые (защитные) поднесущие и поднесущие с вырожденной оценкой канала ($|H[k]| < 10^{-6}$) принудительно обнуляются во избежание деления на близкое к нулю число.

На выходе из каждого символа исключаются пилотные и нулевые поднесущие — в выходной буфер попадают только эквализованные символы данных.

Вот как выглядит работа CFO и эквализации между 2-мя Adalm Pluto SDR:

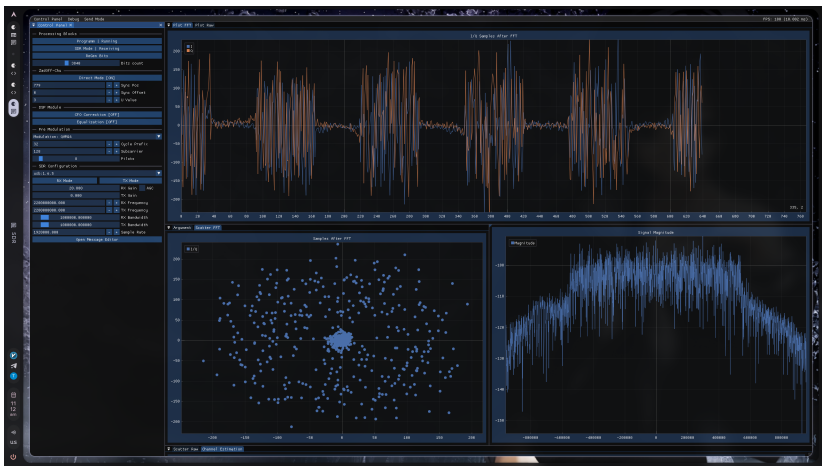


Рис. 40 — Передача без CFO и EQ

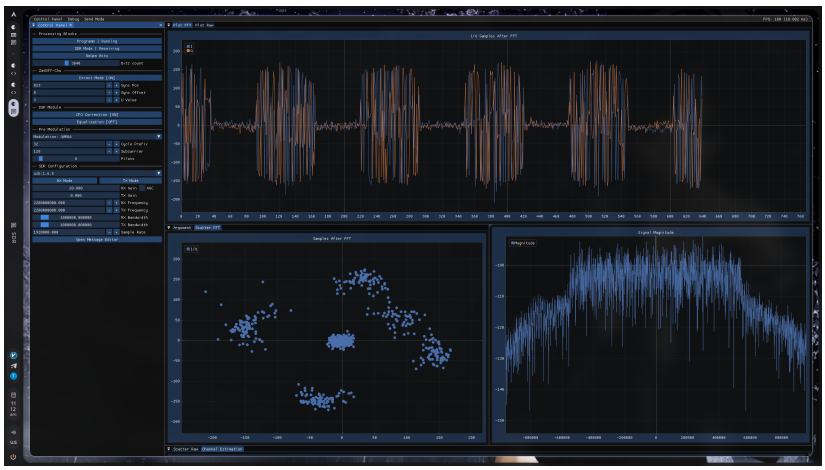


Рис. 41 — Передача с CFO, но без EQ

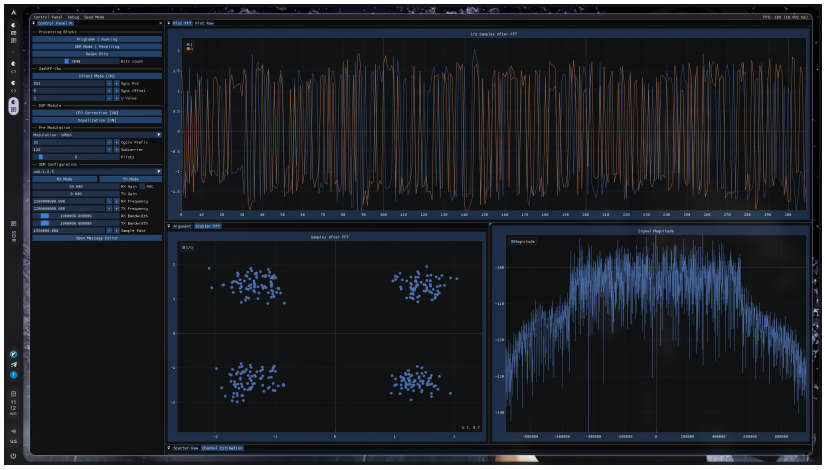


Рис. 42 — Передача с CFO и EQ

Результаты практики

Была написана программа, которая реализует полноценную передачу OFDM символов, с CFO и EQ. Поддерживаемые модуляции:

- BPSK
- QPSK
- QAM16
- QAM64
- QAM256

Также была создана панель с отладочной информация, которая показывает:

- Задержку dsp уровня, которая показывает сколько милисекунд занимает обрабтка сэмплов
- График с пиком корреляции Zadoff-Chu
- График нахождения пиков Zadoff-Chu
- График коррекции CFO

– График оценки канала, которая берется из EQ

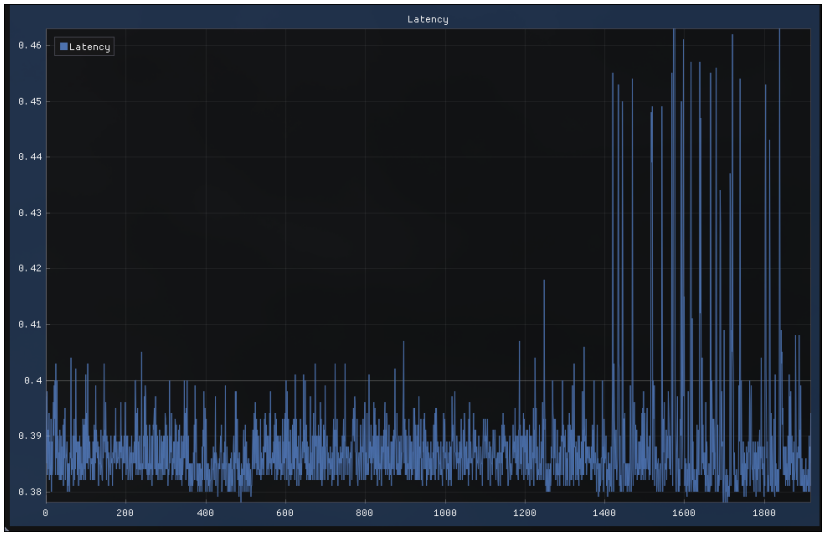


Рис. 43 — Задержка DSP уровня

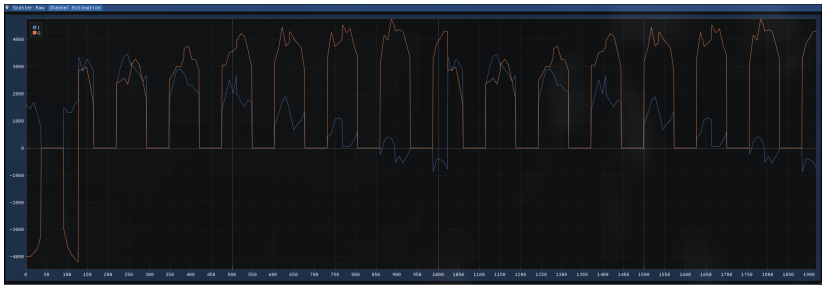


Рис. 44 — Оценка канала

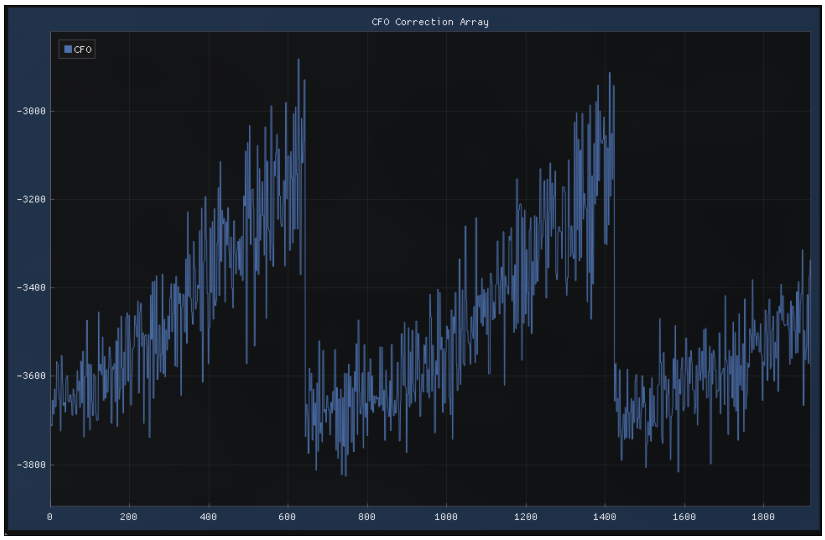


Рис. 45 — График CFO

Пример передачи QAM16 между 2-х Adalm Pluto SDR



Рис. 46 — Передача QAM16(2 SDR)

Вывод

В ходе практики была реализована полная цепочка передачи и приёма OFDM-сигнала: формирование символов из потока данных с поддержкой модуляций BPSK, QPSK, QAM16, QAM64 и QAM256, детектирование границ кадра по корреляции с последовательностью Задова-Чу, оценка и коррекция частотного смещения по циклическому префиксу, оценка канала и эквализация по пилотным поднесущим.

Работоспособность реализованных алгоритмов подтверждена не только на модельных данных, но и на реальном радиоканале между двумя SDR Adalm Pluto — сигнал с модуляцией QAM16 был успешно передан, принят и демодулирован без ошибок, что отражено на приведённых графиках оценки канала и коррекции CFO.

Отдельно была разработана отладочная панель, отображающая ключевые метрики DSP-цепочки в реальном времени: задержку обработки на уровне DSP, пик корреляции и положение синхронизирующей последовательности Задова-Чу, динамику коррекции CFO и текущую оценку канала. Эти инструменты подтвердили, что каждый этап обработки — синхронизация, коррекция частоты, эквализация — работает согласованно и вносит измеримый, ожидаемый вклад в качество приёма.

Практика позволила перейти от теоретического понимания OFDM к работающей системе, где каждый этап обработки сигнала — от формирования символа до восстановления данных — реализован, протестирован и проверен на реальном оборудовании.